

Integrantes do grupo:

Giovanni Castro → RA: 10435745

Yan Andreotti → RA: 10439766

Mateus Kage Moya → RA: 10332608

João Vitor Mintz → RA: 10440421

1. Introdução

Este relatório apresenta o desenvolvimento e a análise de desempenho de um analisador de logs de acesso inspirado em cenários de CDN (Content Delivery Network), com foco na comparação entre estratégias de sincronização em OpenMP. O sistema processa os arquivos `log_distribuido.txt` e `log_concorrente.txt` a partir das URLs previamente carregadas de `manifest.txt`, contabilizando os acessos por meio de uma tabela hash construída sequencialmente e, posteriormente, acessada de forma concorrente.

Para a realização dos experimentos, foram desenvolvidas quatro versões do analisador: `analyzer_seq`, que realiza a execução sequencial; `analyzer_par_atomic`, que utiliza atualizações atômicas; `analyzer_par_critical`, que emprega região crítica global; e `analyzer_par_lock`, que aplica travas por bucket da tabela hash.

A comparação entre essas versões busca avaliar impacto entre diferentes granularidades de sincronização sobre métricas como tempo de execução, speedup, contenção e comportamento do cache.

2. Versão sequencial

2.1. Metodologia

A abordagem sequencial realiza as seguintes etapas: Inicialmente, é validado os argumentos de entrada e os arquivos `manifest.txt` e o log informado na linha de comando são abertos.

Após a abertura dos arquivos, o pipeline segue para a função `insert_in_ht()`, onde é executado em um loop responsável por inserir cada linha presente no arquivo `manifest.txt` para dentro de `ht`, com `hit_count` inicializado em zero. Depois de preenchida a hashtable com os as linhas de `manifest.txt`, o pipeline segue para a função `extrair_urls()`, onde é executado novamente um loop que processa cada linha para extrair trecho necessário para a comparação e, em seguida, incrementa em um o `hit_count` correspondente na tabela.

Após o processamento, é gerado o arquivo `results.csv` que salva os dados presentes na hashtable. Em seguida, é calculado o tempo total de execução do processamento, finalizando com o fechamento dos arquivos e a destruição da hashtable.

2.2. Ambiente Experimental

Máquina Usada: Máquina Linux Ubuntu 22.04.5 LTS, Core i7-1255U, 12 threads, 8GB RAM

Compilador: GCC e flag `-O2`

Ferramentas: `time`, `cachegrind`

2.3. Resultados e testes

Para o log_concorrente.txt:

```
aluno@205:~/Downloads/Projeto2-Computacao-Paralela$ time ./analyzer_seq log_concorrente.txt
Iniciando análise sequencial:
Tempo: 1.4059 segundos

real    0m1,411s
user    0m1,214s
sys     0m0,168s
aluno@205:~/Downloads/Projeto2-Computacao-Paralela$ sort results.csv > results_concorrente.csv
aluno@205:~/Downloads/Projeto2-Computacao-Paralela$ diff -s results_concorrente_seq.csv gabarito_concorrente.csv
Os arquivos results_concorrente_seq.csv e gabarito_concorrente.csv são idênticos
aluno@205:~/Downloads/Projeto2-Computacao-Paralela$ date
ter 19 mai 2026 11:29:55 -03
```

```
(base) aluno@310:~/Downloads/Projeto2-Computacao-Paralela$ valgrind -s --tool=cachegrind ./analyzer_seq log_concorrente.txt
==25676== Cachegrind, a cache and branch-prediction profiler
==25676== Copyright (C) 2002-2017, and GNU GPL'd, by Nicholas Nethercote et al.
==25676== Using Valgrind-3.18.1 and LibVEX; rerun with -h for copyright info
==25676== Command: ./analyzer_seq log_concorrente.txt
==25676==
--25676-- warning: L3 cache found, using its data for the LL simulation.
Iniciando análise sequencial:
Tempo: 45.1464 segundos
==25676==
==25676== I refs:      20,220,410,930
==25676== I1 misses:    1,750
==25676== L1i misses:   1,738
==25676== I1 miss rate: 0.00%
==25676== L1i miss rate: 0.00%
==25676==
==25676== D refs:      7,130,584,442 (4,540,875,184 rd + 2,589,709,258 wr)
==25676== D1 misses:    4,391,539 ( 4,261,960 rd +      129,579 wr)
==25676== L1d misses:   139,748 (   14,064 rd +      125,684 wr)
==25676== D1 miss rate: 0.1% (    0.1% +    0.0% )
==25676== L1d miss rate: 0.0% (    0.0% +    0.0% )
==25676==
==25676== LL refs:      4,393,289 ( 4,263,710 rd +      129,579 wr)
==25676== LL misses:    141,486 (   15,802 rd +      125,684 wr)
==25676== LL miss rate: 0.0% (    0.0% +    0.0% )
(base) aluno@310:~/Downloads/Projeto2-Computacao-Paralela$ sort results.csv > results_seq_concorrente.csv
(base) aluno@310:~/Downloads/Projeto2-Computacao-Paralela$ diff -s results_seq_concorrente.csv gabarito_concorrente.csv
Os arquivos results_seq_concorrente.csv e gabarito_concorrente.csv são idênticos
(base) aluno@310:~/Downloads/Projeto2-Computacao-Paralela$ date
qua 20 mai 2026 08:28:35 -03
```

Para o log_distribuido.txt:

```
aluno@205:~/Downloads/Projeto2-Computacao-Paralela$ time ./analyzer_seq log_distribuido.txt
Iniciando análise sequencial:
Tempo: 2.4910 segundos

real    0m2,496s
user    0m1,817s
sys     0m0,294s
aluno@205:~/Downloads/Projeto2-Computacao-Paralela$ sort results.csv > results_distribuido_seq.csv
aluno@205:~/Downloads/Projeto2-Computacao-Paralela$ diff -s results_distribuido_seq.csv gabarito_distribuido.csv
Os arquivos results_distribuido_seq.csv e gabarito_distribuido.csv são idênticos
aluno@205:~/Downloads/Projeto2-Computacao-Paralela$ date
ter 19 mai 2026 11:32:48 -03
```

```
(base) aluno@310:~/Downloads/Projeto2-Computacao-Paralela$ valgrind -s --tool=cachegrind ./analyzer_seq log_distribuido.txt
==24999== Cachegrind, a cache and branch-prediction profiler
==24999== Copyright (C) 2002-2017, and GNU GPL'd, by Nicholas Nethercote et al.
==24999== Using Valgrind-3.18.1 and LibVEX; rerun with -h for copyright info
==24999== Command: ./analyzer_seq log_distribuido.txt
==24999==
--24999-- warning: L3 cache found, using its data for the LL simulation.
Iniciando análise sequencial:
Tempo: 45.0203 segundos
==24999==
==24999== I refs:      19,865,073,209
==24999== I1 misses:    1,750
==24999== L1i misses:   1,738
==24999== I1 miss rate: 0.00%
==24999== L1i miss rate: 0.00%
==24999==
==24999== D refs:      7,014,788,943 (4,456,522,518 rd + 2,558,266,425 wr)
==24999== D1 misses:    37,213,051 ( 37,080,085 rd +      132,966 wr)
==24999== L1d misses:   139,748 (   14,064 rd +      125,684 wr)
==24999== D1 miss rate: 0.5% (    0.8% +    0.0% )
==24999== L1d miss rate: 0.0% (    0.0% +    0.0% )
==24999==
==24999== LL refs:      37,214,801 ( 37,081,835 rd +      132,966 wr)
==24999== LL misses:    141,486 (   15,802 rd +      125,684 wr)
==24999== LL miss rate: 0.0% (    0.0% +    0.0% )
(base) aluno@310:~/Downloads/Projeto2-Computacao-Paralela$ sort results.csv > results_dist_seq.csv
(base) aluno@310:~/Downloads/Projeto2-Computacao-Paralela$ diff -s results_dist_seq.csv gabarito_distribuido.csv
Os arquivos results_dist_seq.csv e gabarito_distribuido.csv são idênticos
(base) aluno@310:~/Downloads/Projeto2-Computacao-Paralela$ date
qua 20 mai 2026 08:24:31 -03
```

2.4. Tabela

Versão	Dataset	real(s)	user(s)	sys(s)	Tempo(s)
Sequencial	Concorrente	1.411	1.214	0.168	1.4059
Sequencial	Distribuido	2.496	1.817	0.294	2.4910

2.5. Análise Crítica

No caso do `log_concorrente.txt`, a execução levou 1,4059 segundos no tempo medido pelo programa, com `real` em torno de 1,411 segundos, `user` de 1,214 segundos e `sys` de 0,168 segundos. Já no `log_distribuido.txt`, que possui maior distribuição de URLs e que, portanto, deve acessar muitos dados diferentes, levou 2,4910 segundos no tempo medido pelo programa, com `real` em torno de 2,496 segundos, `user` de 1,817 segundos e `sys` de 0,294 segundos. Como esta versão trabalha apenas com uma thread, não há contenção concorrente nem custo de sincronização, portanto, a diferença está relacionada, provavelmente, ao comportamento de acesso sequencial aos dados.

Além disso, os valores de `user` e `sys` indicam que a maior parte do tempo foi consumida em execução em espaço de usuário, principalmente pela realização de leitura de arquivo, parsing de linhas e consultas repetidas à tabela hash. Apesar do aumento no tempo de sistema, ele permaneceu relativamente baixo, indicando que a sobrecarga do kernel foi pequena em comparação ao trabalho computacional do processo.

Como hipótese, o maior tempo observado no `log_distribuido.txt` pode estar relacionado a uma pior localidade de cache, já que a distribuição mais espalhada das URLs pode exigir acessos a regiões mais dispersas da tabela hash. Essa hipótese se fortalece com a análise do `Cachegrind`, que indicou aproximadamente 37,2 milhões de misses em D1 no cenário distribuído, contra cerca de 4,39 milhões no cenário concorrente, sugerindo maior pressão sobre a cache e maior custo de acesso aos dados.

2.6. Conclusão

A versão sequencial apresenta dataset corretamente validado e tempo estável de execução. Os resultados mostram custo principalmente no processamento dos arquivos e consulta da tabela hash, com baixa participação do sistema.

3. Versão Paralela - Atomic

3.1. Metodologia

Com a introdução do paralelismo utilizando a técnica de paralelização do *OpenMP*, utilizaremos como a primeira versão dos testes a versão *Atomic*, baseada na diretiva **#pragma omp atomic update**. Essa abordagem utiliza de operações atômicas fornecidas pelo Hardware para garantir o acesso de múltiplas threads em um único endereçamento sem causar *race conditions* na variável *hit count*. Dessa forma, pode-se reduzir a sobrecarga de sincronização em algumas das regiões críticas, aumentando a capacidade de paralelização na atualização dos contadores na tabela hash.

Para a execução dos testes, utilizamos dois modelos de acesso: o Distribuído e o Concorrente. No modelo Distribuído, as requisições são espalhadas entre URLs diferentes,

reduzindo a contenção das threads. Por outro lado, no modelo Concorrente vemos grande parte dos acessos concentrados em poucos URLs, aumentando a disputa por recursos compartilhados.

3.2. Ambiente Utilizado

Máquina Usada: Máquina Linux Ubuntu 22.04.5 LTS, Core I7-1255U, 12 threads, 8GB RAM

Compilador: GCC e flag -O2

Ferramentas: time, cachegrind

3.3. Atomic

3.3.1. Introdução ao Atomic

Para a execução do *Atomic*, utiliza-se a diretiva **#pragma omp atomic update** para atualizar alguns pontos cruciais no programa e evitar *race conditions*, utilizando operações atômicas fornecidas pelo próprio Hardware, permitindo um paralelismo maior na execução.

3.3.2. Resultados e Testes → Concorrente

Threads: 1

```
aluno@207:~/Downloads/Projeto2Paralela-10435745$ export OMP_NUM_THREADS=1
aluno@207:~/Downloads/Projeto2Paralela-10435745$ gcc -O2 -fopenmp analyzer_par_atomic.c hash_table.c -o analyzer_par_atomic
aluno@207:~/Downloads/Projeto2Paralela-10435745$ time ./analyzer_par_atomic log_concorrente.txt
Iniciando análise Paralelo Atomic:
Tempo: 2.5168 segundos

real    0m2.560s
user    0m1.733s
sys     0m0.623s
aluno@207:~/Downloads/Projeto2Paralela-10435745$ sort results.csv > results_concorrente_sorted.csv
aluno@207:~/Downloads/Projeto2Paralela-10435745$ diff -s results_concorrente_sorted.csv gabarito_concorrente.csv
Os arquivos results_concorrente_sorted.csv e gabarito_concorrente.csv são idênticos
aluno@207:~/Downloads/Projeto2Paralela-10435745$ date
ter 19 mai 2026 11:32:04 -03
aluno@207:~/Downloads/Projeto2Paralela-10435745$
```

Threads: 2

```
aluno@207:~/Downloads/Projeto2Paralela-10435745$ export OMP_NUM_THREADS=2
aluno@207:~/Downloads/Projeto2Paralela-10435745$ gcc -O2 -fopenmp analyzer_par_atomic.c hash_table.c -o analyzer_par_atomic
aluno@207:~/Downloads/Projeto2Paralela-10435745$ time ./analyzer_par_atomic log_concorrente.txt
Iniciando análise Paralelo Atomic:
Tempo: 1.8560 segundos

real    0m1.899s
user    0m1.993s
sys     0m0.539s
aluno@207:~/Downloads/Projeto2Paralela-10435745$ sort results.csv > results_concorrente_sorted.csv
aluno@207:~/Downloads/Projeto2Paralela-10435745$ diff -s results_concorrente_sorted.csv gabarito_concorrente.csv
Os arquivos results_concorrente_sorted.csv e gabarito_concorrente.csv são idênticos
aluno@207:~/Downloads/Projeto2Paralela-10435745$ date
ter 19 mai 2026 11:32:47 -03
aluno@207:~/Downloads/Projeto2Paralela-10435745$
```

Threads: 4

```
aluno@207:~/Downloads/Projeto2Paralela-10435745$ export OMP_NUM_THREADS=4
aluno@207:~/Downloads/Projeto2Paralela-10435745$ time ./analyzer_par_atomic log_concorrente.txt
Iniciando análise Paralelo Atomic:
^[[Tempo: 1.5108 segundos

real    0m1.553s
user    0m2.390s
sys     0m0.443s
aluno@207:~/Downloads/Projeto2Paralela-10435745$ gcc -O2 -fopenmp analyzer_par_atomic.c hash_table.c -o analyzer_par_atomic
aluno@207:~/Downloads/Projeto2Paralela-10435745$ time ./analyzer_par_atomic log_concorrente.txt
Iniciando análise Paralelo Atomic:
^[[Tempo: 1.5168 segundos

real    0m1.554s
user    0m2.391s
sys     0m0.447s
aluno@207:~/Downloads/Projeto2Paralela-10435745$ sort results.csv > results_concorrente_sorted.csv
aluno@207:~/Downloads/Projeto2Paralela-10435745$ diff -s results_concorrente_sorted.csv gabarito_concorrente.csv
Os arquivos results_concorrente_sorted.csv e gabarito_concorrente.csv são idênticos
aluno@207:~/Downloads/Projeto2Paralela-10435745$
```

Threads: 8

```
aluno@207:~/Downloads/Projeto2Paralela-10435745$ export OMP_NUM_THREADS=8
aluno@207:~/Downloads/Projeto2Paralela-10435745$ gcc -O2 -fopenmp analyzer_par_atomic.c hash_table.c -o analyzer_par_atomic
aluno@207:~/Downloads/Projeto2Paralela-10435745$ time ./analyzer_par_atomic log_concorrente.txt
Iniciando análise Paralelo Atomic:
Tempo: 1.3135 segundos

real    0m1,351s
user    0m2,744s
sys     0m0,445s
aluno@207:~/Downloads/Projeto2Paralela-10435745$ sort results.csv > results_concorrente_sorted.csv
aluno@207:~/Downloads/Projeto2Paralela-10435745$ diff -s results_concorrente_sorted.csv gabarito_concorrente.csv
Os arquivos results_concorrente_sorted.csv e gabarito_concorrente.csv são idênticos
aluno@207:~/Downloads/Projeto2Paralela-10435745$ date
ter 19 mai 2026 11:33:59 -03
aluno@207:~/Downloads/Projeto2Paralela-10435745$
```

Execução do Valgrind (4 threads):

```
(base) aluno@309:~/Downloads/Projeto2Paralela-10435745$ valgrind -s --tool=cachegrind ./analyzer_par_atomic log_concorrente.txt
==12386== Cachegrind, a cache and branch-prediction profiler
==12386== Copyright (C) 2002-2017, and GNU GPL'd, by Nicholas Nethercote et al.
==12386== Using Valgrind-3.18.1 and LibVEX; rerun with -h for copyright info
==12386== Command: ./analyzer_par_atomic log_concorrente.txt
==12386==
--12386-- warning: L3 cache found, using its data for the LL simulation.
Iniciando análise Paralelo Atomic:
==12386== brk segment overflow in thread #1: can't grow to 0x485b000
==12386== (see section Limitations in user manual)
==12386== NOTE: further instances of this message will not be shown
Tempo: 55.2177 segundos
==12386==
==12386== I  refs:      25,187,709,232
==12386== I1 misses:    2,198
==12386== LL1 misses:   2,197
==12386== I1 miss rate: 0.00%
==12386== LL1 miss rate: 0.00%
==12386==
==12386== D  refs:      8,844,733,389 (5,646,007,195 rd + 3,198,726,194 wr)
==12386== D1 misses:    70,859,309 ( 49,445,856 rd + 21,413,453 wr)
==12386== LLD misses:   68,968,741 ( 47,601,004 rd + 21,367,737 wr)
==12386== D1 miss rate:  0.8% (    0.9% +    0.7% )
==12386== LLD miss rate: 0.8% (    0.8% +    0.7% )
==12386==
==12386== LL refs:      70,861,507 ( 49,448,054 rd + 21,413,453 wr)
==12386== LL misses:    68,970,938 ( 47,603,201 rd + 21,367,737 wr)
==12386== LL miss rate:  0.2% (    0.2% +    0.7% )
(base) aluno@309:~/Downloads/Projeto2Paralela-10435745$
```

3.3.3. Resultados e Testes → Distribuído

Threads: 1

```
ter 19 mai 2026 11:33:59 -03
aluno@207:~/Downloads/Projeto2Paralela-10435745$ export OMP_NUM_THREADS=1
aluno@207:~/Downloads/Projeto2Paralela-10435745$ gcc -O2 -fopenmp analyzer_par_atomic.c hash_table.c -o analyzer_par_atomic
aluno@207:~/Downloads/Projeto2Paralela-10435745$ time ./analyzer_par_atomic log_distribuido.txt
Iniciando análise Paralelo Atomic:
Tempo: 2.7656 segundos

real    0m2,809s
user    0m2,106s
sys     0m0,598s
aluno@207:~/Downloads/Projeto2Paralela-10435745$ diff -s results_distribuido_sorted.csv gabarito_distribuido.csv
Os arquivos results_distribuido_sorted.csv e gabarito_distribuido.csv são idênticos
aluno@207:~/Downloads/Projeto2Paralela-10435745$ date
ter 19 mai 2026 11:34:59 -03
aluno@207:~/Downloads/Projeto2Paralela-10435745$
```

Threads: 2

```
aluno@207:~/Downloads/Projeto2Paralela-10435745$ export OMP_NUM_THREADS=2
aluno@207:~/Downloads/Projeto2Paralela-10435745$ gcc -O2 -fopenmp analyzer_par_atomic.c hash_table.c -o analyzer_par_atomic
aluno@207:~/Downloads/Projeto2Paralela-10435745$ time ./analyzer_par_atomic log_distribuido.txt
Iniciando análise Paralelo Atomic:
Tempo: 2.0110 segundos

real    0m2,047s
user    0m2,361s
sys     0m0,517s
aluno@207:~/Downloads/Projeto2Paralela-10435745$ sort results.csv > results_concorrente_sorted.csv
aluno@207:~/Downloads/Projeto2Paralela-10435745$ diff -s results_distribuido_sorted.csv gabarito_distribuido.csv
Os arquivos results_distribuido_sorted.csv e gabarito_distribuido.csv são idênticos
aluno@207:~/Downloads/Projeto2Paralela-10435745$ date
ter 19 mai 2026 11:38:11 -03
aluno@207:~/Downloads/Projeto2Paralela-10435745$
```

Threads: 4

```

aluno@207:~/Downloads/Projeto2Paralela-10435745$ export OMP_NUM_THREADS=4
aluno@207:~/Downloads/Projeto2Paralela-10435745$ gcc -O2 -fopenmp analyzer_par_atomic.c hash_table.c -o analyzer_par_atomic
aluno@207:~/Downloads/Projeto2Paralela-10435745$ time ./analyzer_par_atomic log_distribuido.txt
Iniciando análise Paralelo Atomic:
Tempo: 1.7226 segundos

real    0m1,759s
user    0m2,773s
sys     0m0,503s
aluno@207:~/Downloads/Projeto2Paralela-10435745$ sort results.csv > results_concorrente_sorted.csv
aluno@207:~/Downloads/Projeto2Paralela-10435745$ diff -s results_distribuido_sorted.csv gabarito_distribuido.csv
Os arquivos results_distribuido_sorted.csv e gabarito_distribuido.csv são idênticos
aluno@207:~/Downloads/Projeto2Paralela-10435745$ date
ter 19 mai 2026 11:39:29 -03
aluno@207:~/Downloads/Projeto2Paralela-10435745$

```

Threads: 8

```

aluno@207:~/Downloads/Projeto2Paralela-10435745$ export OMP_NUM_THREADS=8
aluno@207:~/Downloads/Projeto2Paralela-10435745$ gcc -O2 -fopenmp analyzer_par_atomic.c hash_table.c -o analyzer_par_atomic
aluno@207:~/Downloads/Projeto2Paralela-10435745$ time ./analyzer_par_atomic log_distribuido.txt
Iniciando análise Paralelo Atomic:
Tempo: 1.3799 segundos

real    0m1,412s
user    0m3,440s
sys     0m0,468s
aluno@207:~/Downloads/Projeto2Paralela-10435745$ sort results.csv > results_concorrente_sorted.csv
aluno@207:~/Downloads/Projeto2Paralela-10435745$ diff -s results_distribuido_sorted.csv gabarito_distribuido.csv
Os arquivos results_distribuido_sorted.csv e gabarito_distribuido.csv são idênticos
aluno@207:~/Downloads/Projeto2Paralela-10435745$ date
ter 19 mai 2026 11:39:57 -03
aluno@207:~/Downloads/Projeto2Paralela-10435745$

```

Execução do Valgrind (4 Threads):

```

(base) aluno@309:~/Downloads/Projeto2Paralela-10435745$ valgrind -s --tool=cachegrind ./analyzer_par_atomic log_distribuido.txt
==12471== Cachegrind, a cache and branch-prediction profiler
==12471== Copyright (C) 2002-2017, and GNU GPL'd, by Nicholas Nethercote et al.
==12471== Using Valgrind-3.18.1 and LibVEX; rerun with -h for copyright info
==12471== Command: ./analyzer_par_atomic log_distribuido.txt
==12471==
--12471-- warning: L3 cache found, using its data for the LL simulation.
Iniciando análise Paralelo Atomic:
==12471== brk segment overflow in thread #1: can't grow to 0x485b000
==12471== (see section Limitations in user manual)
==12471== NOTE: further instances of this message will not be shown
Tempo: 55.7551 segundos
==12471==
==12471== I  refs:      24,682,387,609
==12471== I1 misses:    2,199
==12471== LL1 misses:    2,198
==12471== I1 miss rate: 0.00%
==12471== LL1 miss rate: 0.00%
==12471==
==12471== D  refs:      8,698,750,730 ( 5,525,595,348 rd + 3,173,155,382 wr)
==12471== D1 misses:    103,407,684 ( 81,994,561 rd + 21,413,123 wr)
==12471== LLD misses:    73,833,153 ( 52,465,416 rd + 21,367,737 wr)
==12471== D1 miss rate: 1.2% ( 1.5% + 0.7% )
==12471== LLD miss rate: 0.8% ( 0.9% + 0.7% )
==12471==
==12471== LL refs:      103,409,883 ( 81,996,760 rd + 21,413,123 wr)
==12471== LL misses:    73,835,351 ( 52,467,614 rd + 21,367,737 wr)
==12471== LL miss rate: 0.2% ( 0.2% + 0.7% )
(base) aluno@309:~/Downloads/Projeto2Paralela-10435745$

```

3.3.4. Tabela e Resultados (ambos)

- Tabela Concorrente

Threads	Tempo* (s)	Speedup S = $T(1)/T(p)$	Eficiência E = S/p
1	2,5168	1	1
2	1,8560	1,3560	0,678
4	1,5108	1,6658	0,416
8	1,3135	1,9161	0,239

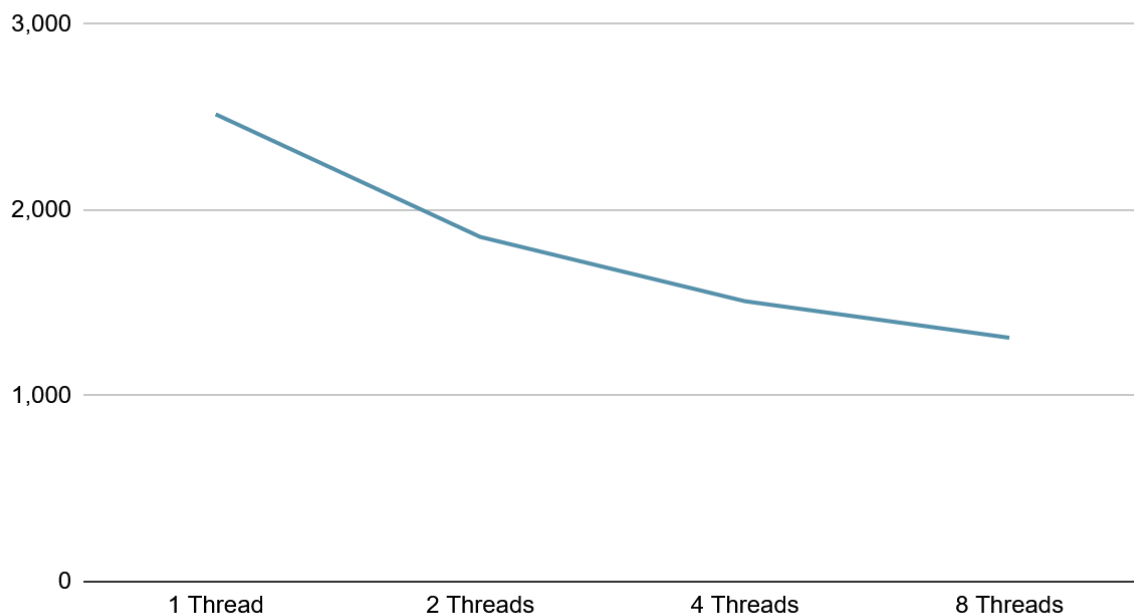
- Tabela Distribuído

Threads	Tempo* (s)	Speedup S = $T(1)/T(p)$	Eficiência E = S/p
1	2,7656	1	1
2	2,0110	1,3752	0,876
4	1,7226	1,6054	0,401
8	1,3799	2,0042	0,25

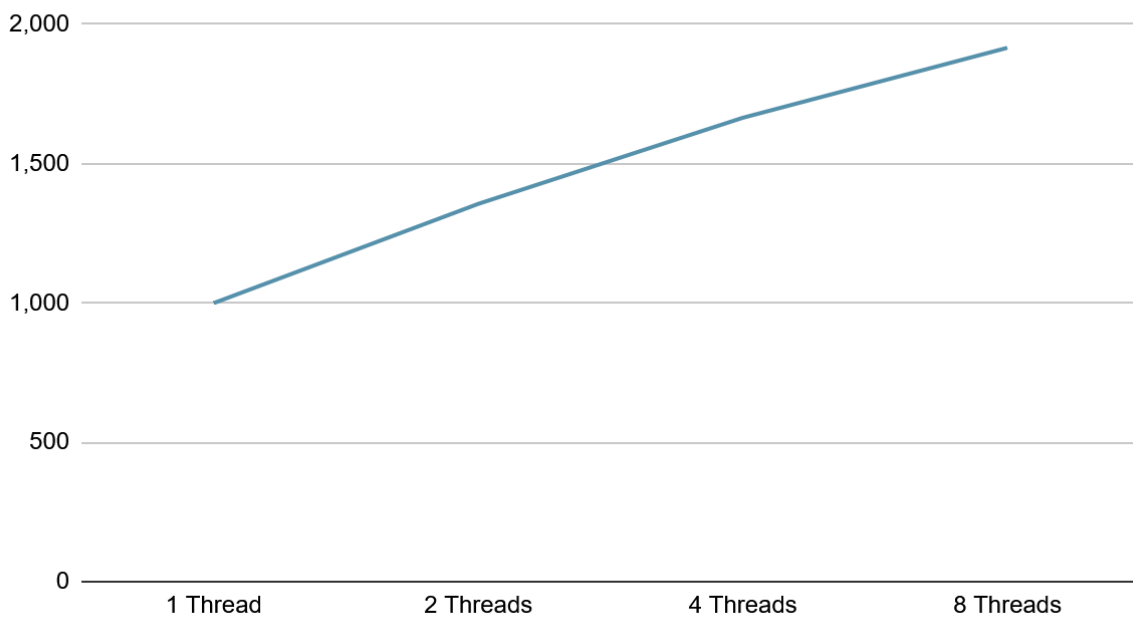
3.3.5. Gráficos (ambos)

- Concorrente *Atomic*

Tempo (ms)

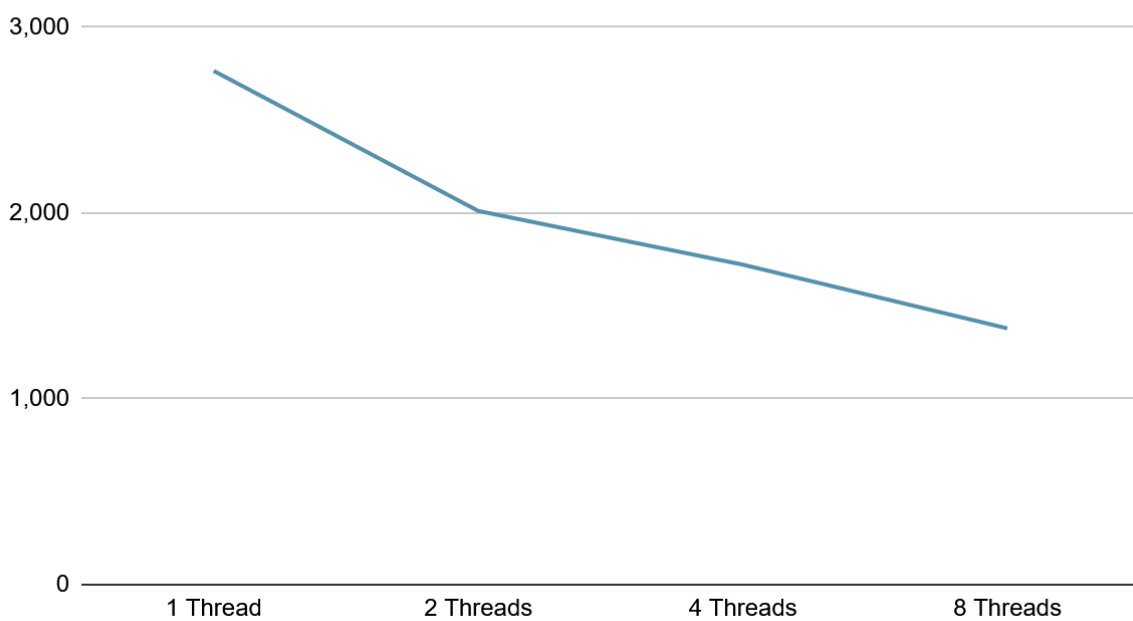


Speedup

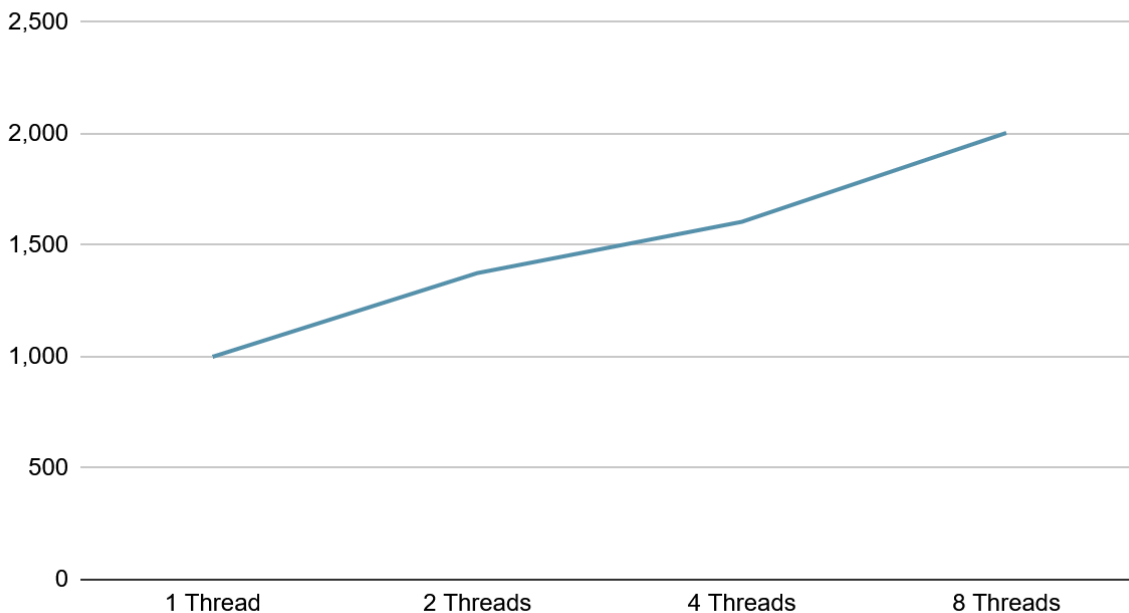


- Distribuído *Atomic*

Tempo (ms)



Speedup



3.3.6. Análise Crítica

Ao utilizar o método Atomic na estratégia de sincronização, observa-se um ganho de desempenho com a paralelização, conforme indicam os valores de speedup. No entanto, há uma perda de eficiência significativa para ambos os modelos (distribuído e concorrente) à medida que o número de threads aumenta, evidenciando o impacto do overhead de sincronização. Apesar da técnica Atomic garantir consistência de dados, ela se torna um gargalo à medida que o paralelismo cresce.

3.3.7. Conclusão *Atomic*

Os resultados mostram, portanto, que o método Atomic permite ganho de desempenho com a paralelização, mas reduz a eficiência à medida que o número de threads aumenta. Entre os dois modelos analisados, o concorrente apresentou menor tempo de execução, enquanto o distribuído apresentou melhor comportamento em termos de speedup e eficiência. Assim, conclui-se que o uso de Atomic garante consistência, porém pode limitar a escalabilidade do sistema.

3.4. *Atomic Padded*

3.4.1. Introdução ao *Atomic Padded*

A versão *Atomic Padded* mantém a utilização da diretiva **#pragma omp atomic update**, porém adiciona preenchimento (*padding*) à estrutura CacheNode. Essa abordagem busca reduzir os efeitos de *false sharing*, fenômeno causado quando múltiplas threads acessam variáveis diferentes localizadas na mesma linha de cache, aumentando a contenção e o tráfego de coerência do cache entre os núcleos do processador.

3.4.2. Execução e Testes → Concorrente

Threads: 1

```

(base) aluno@309:~/Downloads/Projeto2Paralela-10435745$ export OMP_NUM_THREADS=1
(base) aluno@309:~/Downloads/Projeto2Paralela-10435745$ gcc -O2 -fopenmp analyzer_par_atomic_padded.c hash_table.c -o analyzer_par_atomic_padded
(base) aluno@309:~/Downloads/Projeto2Paralela-10435745$ time ./analyzer_par_atomic_padded log_concorrente.txt
Iniciando análise Paralelo Atomic:
Tempo: 2.1814 segundos

real    0m2.215s
user    0m1.764s
sys      0m0.449s
(base) aluno@309:~/Downloads/Projeto2Paralela-10435745$ sort results.csv > results_concorrente_sorted.csv
(base) aluno@309:~/Downloads/Projeto2Paralela-10435745$ diff -s results_concorrente_sorted.csv gabarito_concorrente.csv
Os arquivos results_concorrente_sorted.csv e gabarito_concorrente.csv são idênticos
(base) aluno@309:~/Downloads/Projeto2Paralela-10435745$ date
qua 20 mai 2026 07:41:31 -03
(base) aluno@309:~/Downloads/Projeto2Paralela-10435745$

```

Threads: 2

```

(base) aluno@309:~/Downloads/Projeto2Paralela-10435745$ export OMP_NUM_THREADS=2
(base) aluno@309:~/Downloads/Projeto2Paralela-10435745$ gcc -O2 -fopenmp analyzer_par_atomic_padded.c hash_table.c -o analyzer_par_atomic_padded
(base) aluno@309:~/Downloads/Projeto2Paralela-10435745$ time ./analyzer_par_atomic_padded log_concorrente.txt
Iniciando análise Paralelo Atomic:
Tempo: 1.7976 segundos

real    0m1.857s
user    0m2.097s
sys      0m0.461s
(base) aluno@309:~/Downloads/Projeto2Paralela-10435745$ sort results.csv > results_concorrente_sorted.csv
(base) aluno@309:~/Downloads/Projeto2Paralela-10435745$ diff -s results_concorrente_sorted.csv gabarito_concorrente.csv
Os arquivos results_concorrente_sorted.csv e gabarito_concorrente.csv são idênticos
(base) aluno@309:~/Downloads/Projeto2Paralela-10435745$ date
qua 20 mai 2026 07:43:29 -03
(base) aluno@309:~/Downloads/Projeto2Paralela-10435745$

```

Threads: 4

```

(base) aluno@309:~/Downloads/Projeto2Paralela-10435745$ export OMP_NUM_THREADS=4
(base) aluno@309:~/Downloads/Projeto2Paralela-10435745$ gcc -O2 -fopenmp analyzer_par_atomic_padded.c hash_table.c -o analyzer_par_atomic_padded
(base) aluno@309:~/Downloads/Projeto2Paralela-10435745$ time ./analyzer_par_atomic_padded log_concorrente.txt
Iniciando análise Paralelo Atomic:
Tempo: 1.5599 segundos

real    0m1.593s
user    0m2.494s
sys      0m0.434s
(base) aluno@309:~/Downloads/Projeto2Paralela-10435745$ sort results.csv > results_concorrente_sorted.csv
(base) aluno@309:~/Downloads/Projeto2Paralela-10435745$ diff -s results_concorrente_sorted.csv gabarito_concorrente.csv
Os arquivos results_concorrente_sorted.csv e gabarito_concorrente.csv são idênticos
(base) aluno@309:~/Downloads/Projeto2Paralela-10435745$ date
qua 20 mai 2026 07:45:37 -03
(base) aluno@309:~/Downloads/Projeto2Paralela-10435745$

```

Threads: 8

```

(base) aluno@309:~/Downloads/Projeto2Paralela-10435745$ export OMP_NUM_THREADS=8
(base) aluno@309:~/Downloads/Projeto2Paralela-10435745$ gcc -O2 -fopenmp analyzer_par_atomic_padded.c hash_table.c -o analyzer_par_atomic_padded
(base) aluno@309:~/Downloads/Projeto2Paralela-10435745$ time ./analyzer_par_atomic_padded log_concorrente.txt
Iniciando análise Paralelo Atomic:
Tempo: 1.3275 segundos

real    0m1.364s
user    0m2.828s
sys      0m0.397s
(base) aluno@309:~/Downloads/Projeto2Paralela-10435745$ sort results.csv > results_concorrente_sorted.csv
(base) aluno@309:~/Downloads/Projeto2Paralela-10435745$ diff -s results_concorrente_sorted.csv gabarito_concorrente.csv
Os arquivos results_concorrente_sorted.csv e gabarito_concorrente.csv são idênticos
(base) aluno@309:~/Downloads/Projeto2Paralela-10435745$ date
qua 20 mai 2026 07:46:03 -03
(base) aluno@309:~/Downloads/Projeto2Paralela-10435745$

```

Execução do Valgrind (4 Threads):

```
(base) aluno@309:~/Downloads/Projeto2Paralela-10435745$ valgrind -s --tool=cachegrind ./analyzer_par_atomic_padded log_concorrente.txt
==11827== Cachegrind, a cache and branch-prediction profiler
==11827== Copyright (C) 2002-2017, and GNU GPL'd, by Nicholas Nethercote et al.
==11827== Using Valgrind-3.18.1 and LibVEX; rerun with -h for copyright info
==11827== Command: ./analyzer_par_atomic_padded log_concorrente.txt
==11827==
--11827-- warning: L3 cache found, using its data for the LL simulation.
Iniciando análise Paralelo Atomic:
==11827== brk segment overflow in thread #1: can't grow to 0x485b000
==11827== (see section Limitations in user manual)
==11827== NOTE: further instances of this message will not be shown
Tempo: 56.4338 segundos
==11827==
==11827== I refs:      25,262,628,372
==11827== Ii misses:    2,446
==11827== Lli misses:   2,403
==11827== Ii miss rate: 0.00%
==11827== Lli miss rate: 0.00%
==11827==
==11827== D refs:      8,861,218,750 (5,662,492,559 rd + 3,198,726,191 wr)
==11827== D1 misses:   70,815,947 ( 49,406,628 rd + 21,409,319 wr)
==11827== Lld misses: 68,968,061 ( 47,600,324 rd + 21,367,737 wr)
==11827== D1 miss rate: 0.8% ( 0.9% + 0.7% )
==11827== Lld miss rate: 0.8% ( 0.8% + 0.7% )
==11827==
==11827== LL refs:      70,818,393 ( 49,409,074 rd + 21,409,319 wr)
==11827== LL misses:   68,970,464 ( 47,602,727 rd + 21,367,737 wr)
==11827== LL miss rate: 0.2% ( 0.2% + 0.7% )
(base) aluno@309:~/Downloads/Projeto2Paralela-10435745$
```

3.4.3. Execução e Testes → Distribuído

Threads: 1

```
aluno@207:~/Downloads/Projeto2Paralela-10435745$ export OMP_NUM_THREADS=1
aluno@207:~/Downloads/Projeto2Paralela-10435745$ gcc -O2 -fopenmp analyzer_par_atomic_padded.c hash_table.c -o analyzer_par_atomic_padded
aluno@207:~/Downloads/Projeto2Paralela-10435745$ time ./analyzer_par_atomic_padded log_distribuido.txt
Iniciando análise Paralelo Atomic:
Tempo: 2.7808 segundos

real    0m2.819s
user    0m2.195s
sys      0m0.599s
aluno@207:~/Downloads/Projeto2Paralela-10435745$ sort results.csv > results_concorrente_sorted.csv
aluno@207:~/Downloads/Projeto2Paralela-10435745$ diff -s results_distribuido_sorted.csv gabarito_distribuido.csv
Os arquivos results_distribuido_sorted.csv e gabarito_distribuido.csv são idênticos
aluno@207:~/Downloads/Projeto2Paralela-10435745$ date
ter 19 mai 2026 11:47:27 -03
aluno@207:~/Downloads/Projeto2Paralela-10435745$
```

Threads: 2

```
aluno@207:~/Downloads/Projeto2Paralela-10435745$ export OMP_NUM_THREADS=2
aluno@207:~/Downloads/Projeto2Paralela-10435745$ gcc -O2 -fopenmp analyzer_par_atomic_padded.c hash_table.c -o analyzer_par_atomic_padded
aluno@207:~/Downloads/Projeto2Paralela-10435745$ time ./analyzer_par_atomic_padded log_distribuido.txt
Iniciando análise Paralelo Atomic:
Tempo: 2.1327 segundos

real    0m2.174s
user    0m2.629s
sys      0m0.533s
aluno@207:~/Downloads/Projeto2Paralela-10435745$ sort results.csv > results_concorrente_sorted.csv
aluno@207:~/Downloads/Projeto2Paralela-10435745$ diff -s results_distribuido_sorted.csv gabarito_distribuido.csv
Os arquivos results_distribuido_sorted.csv e gabarito_distribuido.csv são idênticos
aluno@207:~/Downloads/Projeto2Paralela-10435745$ date
ter 19 mai 2026 11:48:06 -03
aluno@207:~/Downloads/Projeto2Paralela-10435745$
```

Threads: 4

```
aluno@207:~/Downloads/Projeto2Paralela-10435745$ export OMP_NUM_THREADS=4
aluno@207:~/Downloads/Projeto2Paralela-10435745$ gcc -O2 -fopenmp analyzer_par_atomic_padded.c hash_table.c -o analyzer_par_atomic_padded
aluno@207:~/Downloads/Projeto2Paralela-10435745$ time ./analyzer_par_atomic_padded log_distribuido.txt
Iniciando análise Paralelo Atomic:
Tempo: 1.6340 segundos

real    0m1.683s
user    0m2.732s
sys      0m0.476s
aluno@207:~/Downloads/Projeto2Paralela-10435745$ sort results.csv > results_concorrente_sorted.csv
aluno@207:~/Downloads/Projeto2Paralela-10435745$ diff -s results_distribuido_sorted.csv gabarito_distribuido.csv
Os arquivos results_distribuido_sorted.csv e gabarito_distribuido.csv são idênticos
aluno@207:~/Downloads/Projeto2Paralela-10435745$ date
ter 19 mai 2026 11:48:33 -03
aluno@207:~/Downloads/Projeto2Paralela-10435745$
```

Threads: 8

```

aluno@207:~/Downloads/Projeto2Paralela-10435745$ export OMP_NUM_THREADS=8
aluno@207:~/Downloads/Projeto2Paralela-10435745$ gcc -O2 -fopenmp analyzer_par_atomic_padded.c hash_table.c -o analyzer_par_atomic_padded
aluno@207:~/Downloads/Projeto2Paralela-10435745$ time ./analyzer_par_atomic_padded log_distribuido.txt
Iniciando análise Paralelo Atomic:
Tempo: 1.4065 segundos

real    0m1.451s
user    0m3.061s
sys      0m0.459s
aluno@207:~/Downloads/Projeto2Paralela-10435745$ sort results.csv > results_concorrente_sorted.csv
aluno@207:~/Downloads/Projeto2Paralela-10435745$ diff -s results_distribuido_sorted.csv gabarito_distribuido.csv
Os arquivos results_distribuido_sorted.csv e gabarito_distribuido.csv são idênticos
aluno@207:~/Downloads/Projeto2Paralela-10435745$ date
ter 19 mai 2026 11:48:53 -03
aluno@207:~/Downloads/Projeto2Paralela-10435745$

```

Execução do Valgrind (4 Threads):

```

(base) aluno@309:~/Downloads/Projeto2Paralela-10435745$ valgrind -s --tool=cachegrind ./analyzer_par_atomic_padded log_distribuido.txt
==12052== Cachegrind, a cache and branch-prediction profiler
==12052== Copyright (C) 2002-2017, and GNU GPL'd, by Nicholas Nethercote et al.
==12052== Using Valgrind-3.18.1 and LibVEX; rerun with -h for copyright info
==12052== Command: ./analyzer_par_atomic_padded log_distribuido.txt
==12052==
--12052-- warning: L3 cache found, using its data for the LL simulation.
Iniciando análise Paralelo Atomic:
==12052== brk segment overflow in thread #1: can't grow to 0x485b000
==12052== (see section Limitations in user manual)
==12052== NOTE: further instances of this message will not be shown
Tempo: 55.2582 segundos
==12052==
==12052== I  refs:      24,736,275,693
==12052== I1 misses:    2,198
==12052== L1i misses:    2,197
==12052== I1 miss rate:  0.00%
==12052== L1i miss rate: 0.00%
==12052==
==12052== D refs:      8,710,920,248 (5,537,764,863 rd + 3,173,155,385 wr)
==12052== D1 misses:    103,407,966 ( 81,994,821 rd + 21,413,145 wr)
==12052== L1d misses:    73,832,678 ( 52,464,941 rd + 21,367,737 wr)
==12052== D1 miss rate:  1.2% ( 1.5% + 0.7% )
==12052== L1d miss rate: 0.8% ( 0.9% + 0.7% )
==12052==
==12052== LL refs:      103,410,164 ( 81,997,019 rd + 21,413,145 wr)
==12052== LL misses:    73,834,875 ( 52,467,138 rd + 21,367,737 wr)
==12052== LL miss rate:  0.2% ( 0.2% + 0.7% )
(base) aluno@309:~/Downloads/Projeto2Paralela-10435745$

```

3.4.4. Tabelas

- Tabela Concorrente

Threads	Tempo* (s)	Speedup S = $T(1)/T(p)$	Eficiência E = S/p
1	2,1814	1	1
2	1,7976	1,2135	0,6067
4	1,5599	1,3984	0,3496
8	1,3275	1,6432	0,2054

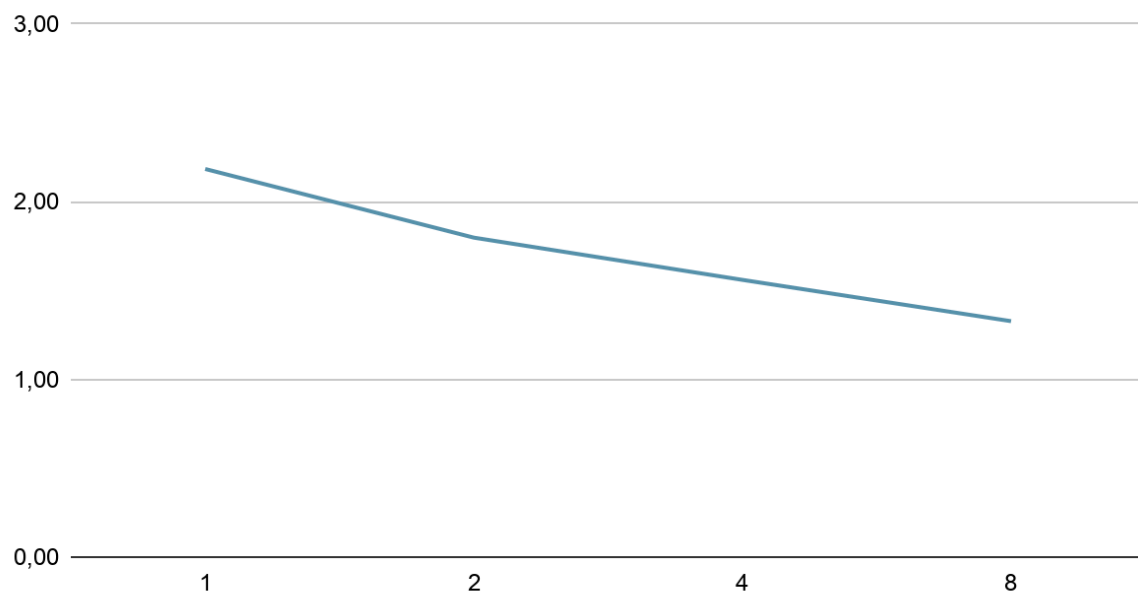
- Tabela Distribuído

Threads	Tempo* (s)	Speedup S = $T(1)/T(p)$	Eficiência E = S/p
1	2,7808	1	1
2	2,1327	1,3038	0,651
4	1,6340	1,7018	0,425
8	1,4065	1,9771	0,247

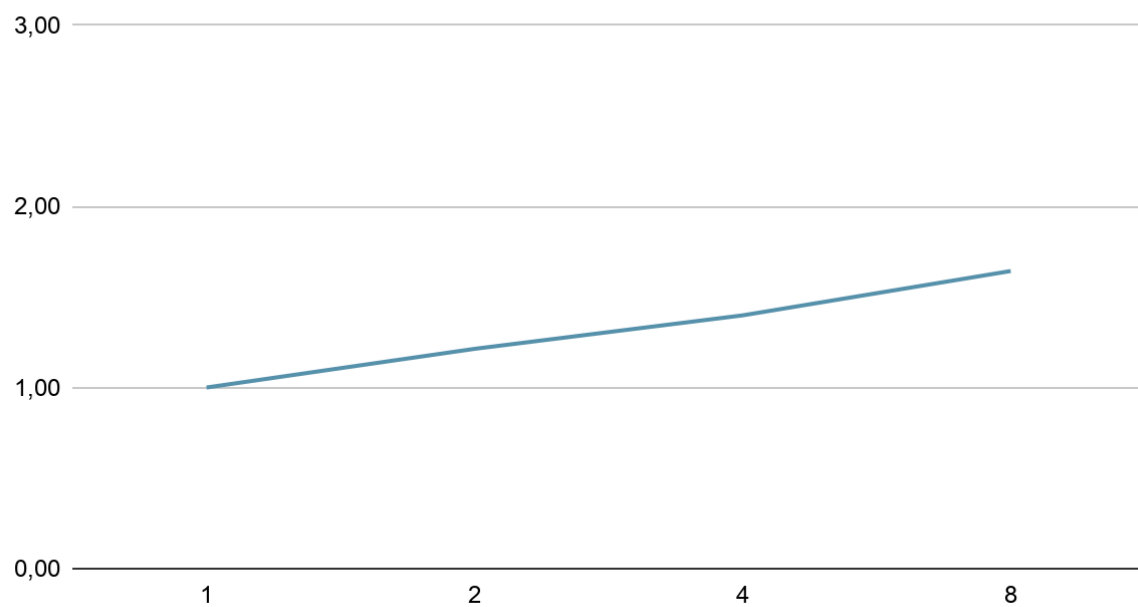
3.4.5. Gráficos

- Concorrente *Atomic Padded*

Tempo (ms)

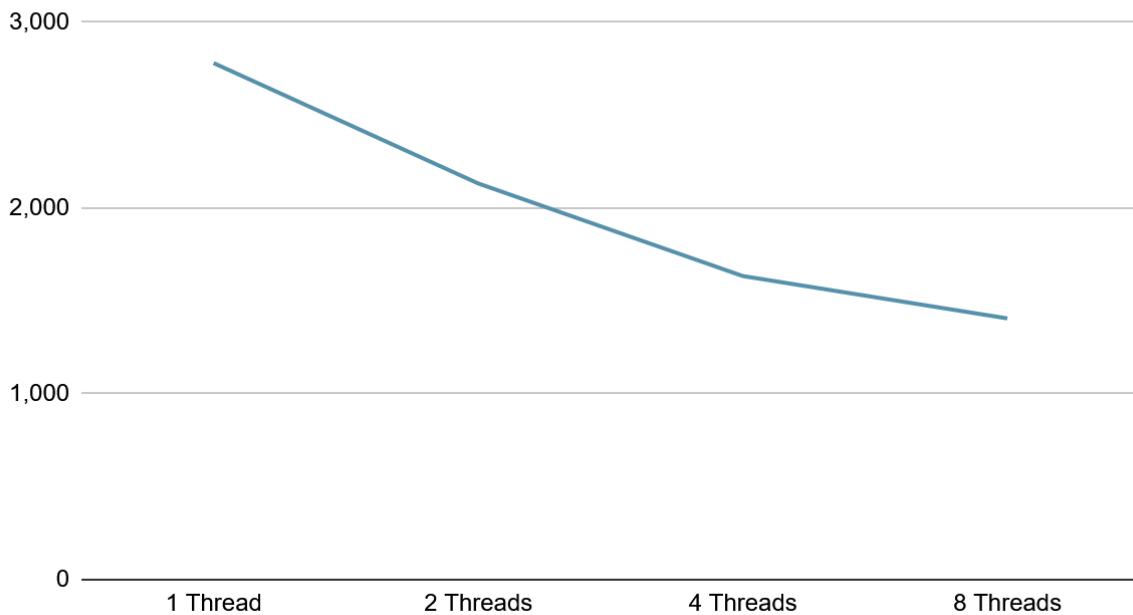


Speedup

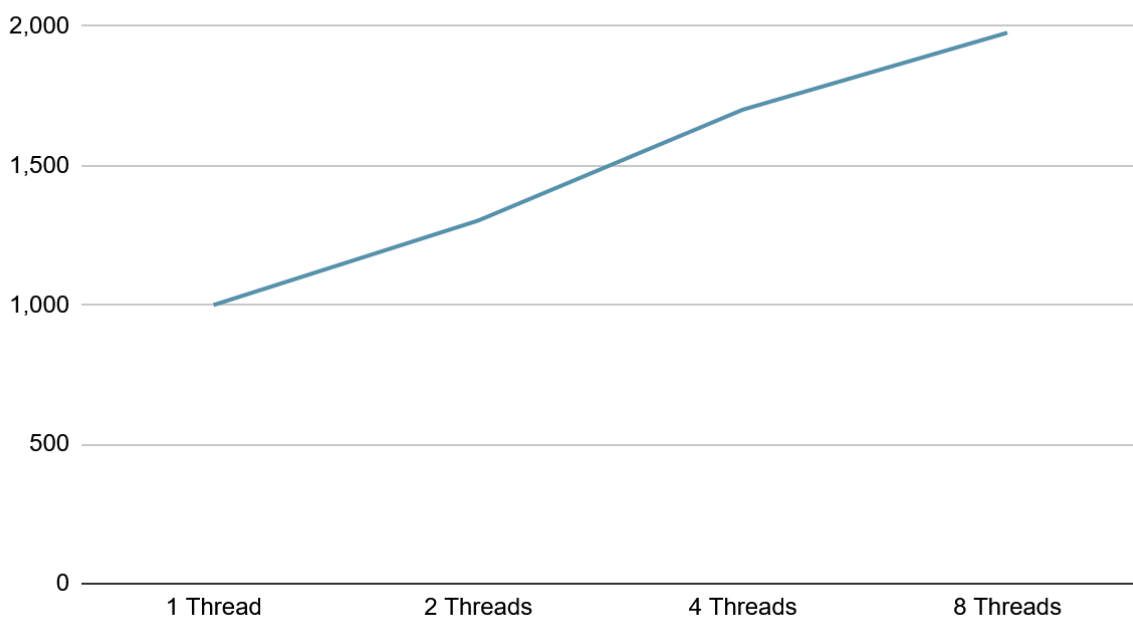


- Distribuído *Atomic Padded*

Tempo (ms)



Speedup



3.4.6. Análise Crítica

O método Atomic Padded apresentou comportamento semelhante à versão Atomic tradicional, mantendo ganhos de desempenho com a paralelização, porém ainda demonstrando queda de eficiência conforme o aumento do número de threads. A utilização de padding reduziu parcialmente os impactos causados pelo compartilhamento de cache entre threads (false sharing), mas não eliminou completamente os custos de sincronização das operações atômicas.

3.4.7. Conclusão *Atomic Padded*

Dessa forma, observa-se que, embora o padding contribua para reduzir contenção em memória, o overhead associado às operações atômicas continua limitando a escalabilidade da aplicação.

3.5. Conclusão Geral

Comparando as versões *Atomic* e *Atomic Padded*, observa-se que ambas apresentaram melhora no tempo de execução em relação à execução sequencial, demonstrando os benefícios do paralelismo com OpenMP. Entretanto, em ambas as abordagens houve perda progressiva de eficiência com o aumento do número de threads, indicando limitações de escalabilidade associadas aos custos de sincronização e à contenção entre acessos concorrentes.

A versão *Padded* apresentou pequenas melhorias relacionadas ao comportamento de cache, reduzindo parcialmente os efeitos de *false sharing*, porém sem eliminar o overhead causado pelas operações atômicas. Assim, conclui-se que o uso de operações atômicas é eficiente para garantir consistência dos dados, mas ainda apresenta limitações em cenários de alta concorrência.

4. Versão Paralela - Critical

4.1. Metodologia

A versão Critical utiliza a diretiva `#pragma omp critical` para proteger o acesso à tabela hash. Essa abordagem cria uma região crítica global, por isso o nome, permitindo que apenas uma thread por vez execute o bloco de incremento do *hit_count*. Ao contrário das operações atômicas, o **#pragma omp critical** é implementado como um mutex do sistema operacional, adequado para blocos de código maiores, mas com overhead superior para operações simples de incremento.

Assim como nas demais versões, foram testados dois modelos de acesso: o Distribuído, em que as requisições se distribuem entre URLs distintas, e o Concorrente, com 90% do tráfego concentrado em apenas 10 URLs, criando alta contenção.

4.2 Ambiente Utilizado

Máquina Usada: Máquina Linux Ubuntu 22.04.5 LTS, Core i7-1255U, 12 threads, 8GB RAM

Compilador: GCC e flag -O2

Ferramentas: time, cachegrind

4.3. Critical

4.3.1. Introdução ao Critical

Na versão Critical, a diretiva `#pragma omp critical` protege o bloco de atualização do *hit_count* na tabela hash. Por ser um portão global, apenas uma thread executa o bloco por

vez, independentemente de qual URL está sendo acessada. Essa granularidade grossa de sincronização reduz o paralelismo efetivo especialmente em cenários de alta contenção.

4.3.2. Execução e Testes - Concorrente

Threads: 1

```
(base) aluno@309:~/Downloads/Projeto2Paralela-10435745$ export OMP_NUM_THREADS=1
(base) aluno@309:~/Downloads/Projeto2Paralela-10435745$ gcc -O2 -fopenmp analyzer_par_critical.c hash_table.c -o analyzer_par_critical
(base) aluno@309:~/Downloads/Projeto2Paralela-10435745$ time ./analyzer_par_critical log_concorrente.txt
Iniciando análise paralela (critical) com 1 thread(s)...
Tempo total: 2.0341 segundos

real    0m2,066s
user    0m1,613s
sys     0m0,443s
(base) aluno@309:~/Downloads/Projeto2Paralela-10435745$ sort results.csv > results_concorrente_sorted.csv
(base) aluno@309:~/Downloads/Projeto2Paralela-10435745$ diff -s results_concorrente_sorted.csv gabarito_concorrente.csv
Os arquivos results_concorrente_sorted.csv e gabarito_concorrente.csv são idênticos
(base) aluno@309:~/Downloads/Projeto2Paralela-10435745$ date
qua 20 mai 2026 08:31:01 -03
```

Threads: 2

```
(base) aluno@309:~/Downloads/Projeto2Paralela-10435745$ export OMP_NUM_THREADS=2
(base) aluno@309:~/Downloads/Projeto2Paralela-10435745$ time ./analyzer_par_critical log_concorrente.txt
Iniciando análise paralela (critical) com 2 thread(s)...
Tempo total: 2.0592 segundos

real    0m2,098s
user    0m2,685s
sys     0m0,465s
(base) aluno@309:~/Downloads/Projeto2Paralela-10435745$ sort results.csv > results_concorrente_sorted.csv
(base) aluno@309:~/Downloads/Projeto2Paralela-10435745$ diff -s results_concorrente_sorted.csv gabarito_concorrente.csv
Os arquivos results_concorrente_sorted.csv e gabarito_concorrente.csv são idênticos
(base) aluno@309:~/Downloads/Projeto2Paralela-10435745$ date
qua 20 mai 2026 08:31:39 -03
(base) aluno@309:~/Downloads/Projeto2Paralela-10435745$
```

Threads: 4

```
(base) aluno@309:~/Downloads/Projeto2Paralela-10435745$ export OMP_NUM_THREADS=4
(base) aluno@309:~/Downloads/Projeto2Paralela-10435745$ time ./analyzer_par_critical log_concorrente.txt
Iniciando análise paralela (critical) com 4 thread(s)...
Tempo total: 1.9400 segundos

real    0m1,973s
user    0m4,721s
sys     0m0,421s
(base) aluno@309:~/Downloads/Projeto2Paralela-10435745$ sort results.csv > results_concorrente_sorted.csv
(base) aluno@309:~/Downloads/Projeto2Paralela-10435745$ diff -s results_concorrente_sorted.csv gabarito_concorrente.csv
Os arquivos results_concorrente_sorted.csv e gabarito_concorrente.csv são idênticos
(base) aluno@309:~/Downloads/Projeto2Paralela-10435745$ date
qua 20 mai 2026 08:32:14 -03
(base) aluno@309:~/Downloads/Projeto2Paralela-10435745$
```

Threads: 8

```
(base) aluno@309:~/Downloads/Projeto2Paralela-10435745$ export OMP_NUM_THREADS=8
(base) aluno@309:~/Downloads/Projeto2Paralela-10435745$ time ./analyzer_par_critical log_concorrente.txt
Iniciando análise paralela (critical) com 8 thread(s)...
Tempo total: 3.0253 segundos

real    0m3,060s
user    0m17,291s
sys     0m0,417s
(base) aluno@309:~/Downloads/Projeto2Paralela-10435745$ sort results.csv > results_concorrente_sorted.csv
(base) aluno@309:~/Downloads/Projeto2Paralela-10435745$ diff -s results_concorrente_sorted.csv gabarito_concorrente.csv
Os arquivos results_concorrente_sorted.csv e gabarito_concorrente.csv são idênticos
(base) aluno@309:~/Downloads/Projeto2Paralela-10435745$ date
qua 20 mai 2026 08:35:15 -03
(base) aluno@309:~/Downloads/Projeto2Paralela-10435745$
```

Execução do Valgrind (4 threads)

```
(base) aluno@309:~/Downloads/Projeto2Paralela-10435745$ valgrind -s --tool=cachegrind ./analyzer_par_critical log_concorrente.txt
==13597== Cachegrind, a cache and branch-prediction profiler
==13597== Copyright (C) 2002-2017, and GNU GPL'd, by Nicholas Nethercote et al.
==13597== Using Valgrind-3.18.1 and LibVEX; rerun with -h for copyright info
==13597== Command: ./analyzer_par_critical log_concorrente.txt
==13597==
--13597-- warning: L3 cache found, using its data for the LL simulation.
Iniciando analise paralela (critical) com 4 thread(s)...
==13597== brk segment overflow in thread #1: can't grow to 0x485b000
==13597== (see section Limitations in user manual)
==13597== NOTE: further instances of this message will not be shown
Tempo total: 57.5001 segundos
==13597==
==13597== I   refs:      25,321,830,848
==13597== I1 misses:      2,271
==13597== LLi misses:      2,270
==13597== I1 miss rate:      0.00%
==13597== LLi miss rate:      0.00%
==13597==
==13597== D   refs:      8,920,956,245 ( 5,702,199,108 rd + 3,218,757,137 wr)
==13597== D1 misses:      57,263,183 ( 35,849,471 rd + 21,413,712 wr)
==13597== LLD misses:      55,207,204 ( 33,865,990 rd + 21,341,214 wr)
==13597== D1 miss rate:      0.6% ( 0.6% + 0.7% )
==13597== LLD miss rate:      0.6% ( 0.6% + 0.7% )
==13597==
==13597== LL refs:      57,265,454 ( 35,851,742 rd + 21,413,712 wr)
==13597== LL misses:      55,209,474 ( 33,868,260 rd + 21,341,214 wr)
==13597== LL miss rate:      0.2% ( 0.1% + 0.7% )
(base) aluno@309:~/Downloads/Projeto2Paralela-10435745$
```

4.3.3. Execução e Testes - Distribuído

Threads: 1

```
(base) aluno@310:~/Downloads/Projeto2-Computacao-Paralela$ export OMP_NUM_THREADS=1
(base) aluno@310:~/Downloads/Projeto2-Computacao-Paralela$ time ./analyzer_par_critical log_distribuido.txt
Iniciando analise paralela (critical) com 1 thread(s)...
Tempo total: 3.0543 segundos

real    0m3,121s
user    0m2,284s
sys     0m0,631s
(base) aluno@310:~/Downloads/Projeto2-Computacao-Paralela$ sort results.csv > results_critical_dist.csv
(base) aluno@310:~/Downloads/Projeto2-Computacao-Paralela$ diff -s results_critical_dist.csv gabarito_distribuido.csv
Os arquivos results_critical_dist.csv e gabarito_distribuido.csv são idênticos
(base) aluno@310:~/Downloads/Projeto2-Computacao-Paralela$ time

real    0m0,000s
user    0m0,000s
sys     0m0,000s
(base) aluno@310:~/Downloads/Projeto2-Computacao-Paralela$ date
qua 20 mai 2026 08:32:30 -03
```

Threads: 2

```
(base) aluno@310:~/Downloads/Projeto2-Computacao-Paralela$ export OMP_NUM_THREADS=2
(base) aluno@310:~/Downloads/Projeto2-Computacao-Paralela$ time ./analyzer_par_critical log_distribuido.txt
Iniciando analise paralela (critical) com 2 thread(s)...
Tempo total: 2.1251 segundos

real    0m2,166s
user    0m2,934s
sys     0m0,428s
(base) aluno@310:~/Downloads/Projeto2-Computacao-Paralela$ sort results.csv > results_critical_dist.csv
(base) aluno@310:~/Downloads/Projeto2-Computacao-Paralela$ diff -s results_critical_dist.csv gabarito_distribuido.csv
Os arquivos results_critical_dist.csv e gabarito_distribuido.csv são idênticos
(base) aluno@310:~/Downloads/Projeto2-Computacao-Paralela$ date
qua 20 mai 2026 08:33:48 -03
```

Threads: 4

```
(base) aluno@310:~/Downloads/Projeto2-Computacao-Paralela$ export OMP_NUM_THREADS=4
(base) aluno@310:~/Downloads/Projeto2-Computacao-Paralela$ time ./analyzer_par_critical log_distribuido.txt
Iniciando analise paralela (critical) com 4 thread(s)...
Tempo total: 1.9962 segundos

real    0m2,026s
user    0m4,929s
sys     0m0,422s
(base) aluno@310:~/Downloads/Projeto2-Computacao-Paralela$ sort results.csv > results_critical_dist.csv
(base) aluno@310:~/Downloads/Projeto2-Computacao-Paralela$ diff -s results_critical_dist.csv gabarito_distribuido.csv
Os arquivos results_critical_dist.csv e gabarito_distribuido.csv são idênticos
(base) aluno@310:~/Downloads/Projeto2-Computacao-Paralela$ date
qua 20 mai 2026 08:34:41 -03
```

Threads: 8

```
(base) aluno@310:~/Downloads/Projeto2-Computacao-Paralela$ export OMP_NUM_THREADS=8
(base) aluno@310:~/Downloads/Projeto2-Computacao-Paralela$ time ./analyzer_par_critical log_distribuido.txt
Iniciando analise paralela (critical) com 8 thread(s)...
Tempo total: 3.1656 segundos

real    0m3,207s
user    0m18,313s
sys     0m0,455s
(base) aluno@310:~/Downloads/Projeto2-Computacao-Paralela$ sort results.csv > results_critical_dist.csv
(base) aluno@310:~/Downloads/Projeto2-Computacao-Paralela$ diff -s results_critical_dist.csv gabarito_distribuido.csv
Os arquivos results_critical_dist.csv e gabarito_distribuido.csv são idênticos
(base) aluno@310:~/Downloads/Projeto2-Computacao-Paralela$ date
qua 20 mai 2026 08:35:30 -03
```

Execução do Valgrind (4 threads)

```
(base) aluno@310:~/Downloads/Projeto2-Computacao-Paralela$ export OMP_NUM_THREADS=4
(base) aluno@310:~/Downloads/Projeto2-Computacao-Paralela$ valgrind -s --tool=cachegrind ./analyzer_par_critical log_distribuido.txt
==26732== Cachegrind, a cache and branch-prediction profiler
==26732== Copyright (c) 2002-2017, and GNU GPL'd, by Nicholas Nethercote et al.
==26732== Using Valgrind-3.18.1 and LibVEX; rerun with -h for copyright info
==26732== Command: ./analyzer_par_critical log_distribuido.txt
==26732==
--26732-- warning: L3 cache found, using its data for the LL simulation.
Iniciando analise paralela (critical) com 4 thread(s)...
==26732== brk segment overflow in thread #1: can't grow to 0x485b000
==26732== (see section Limitations in user manual)
==26732== NOTE: further instances of this message will not be shown
Tempo total: 59.1479 segundos
==26732==
==26732== I refs:      24,960,193,321
==26732== I1 misses:    2,273
==26732== L1i misses:   2,272
==26732== I1 miss rate: 0.00%
==26732== L1i miss rate: 0.00%
==26732==
==26732== D refs:      8,807,202,784 (5,613,688,978 rd + 3,193,513,806 wr)
==26732== D1 misses:   89,641,461 ( 68,228,395 rd + 21,413,066 wr)
==26732== L1d misses:  60,134,932 ( 38,785,132 rd + 21,349,800 wr)
==26732== D1 miss rate: 1.0% ( 1.2% + 0.7% )
==26732== L1d miss rate: 0.7% ( 0.7% + 0.7% )
==26732==
==26732== LL refs:      89,643,734 ( 68,230,668 rd + 21,413,066 wr)
==26732== LL misses:   60,137,204 ( 38,787,404 rd + 21,349,800 wr)
==26732== LL miss rate: 0.2% ( 0.1% + 0.7% )
```

4.3.4. Tabela e Resultados

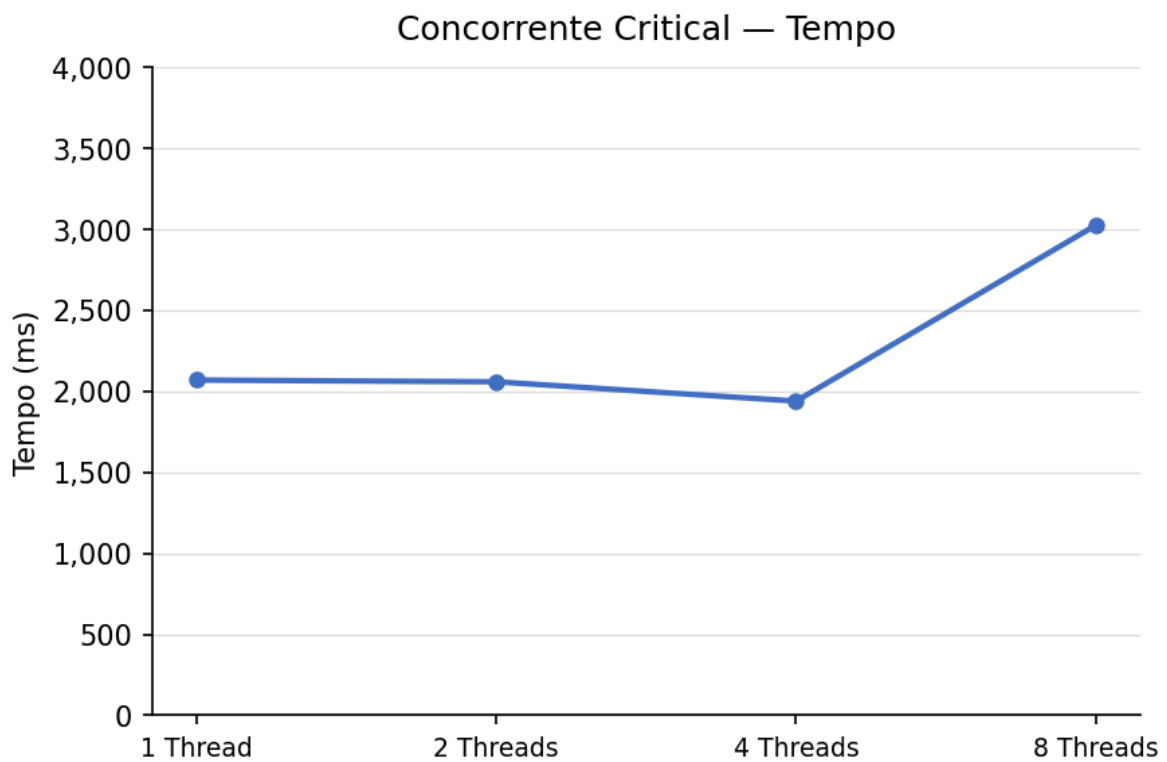
- Tabela Concorrente

Threads	Tempo* (s)	Speedup S = $T(1)/T(p)$	Eficiência E = S/p
1	2,0700	1	1
2	2,0592	1,0052	0,503
4	1,9400	1,0670	0,267
8	3,0253	0,6843	0,086

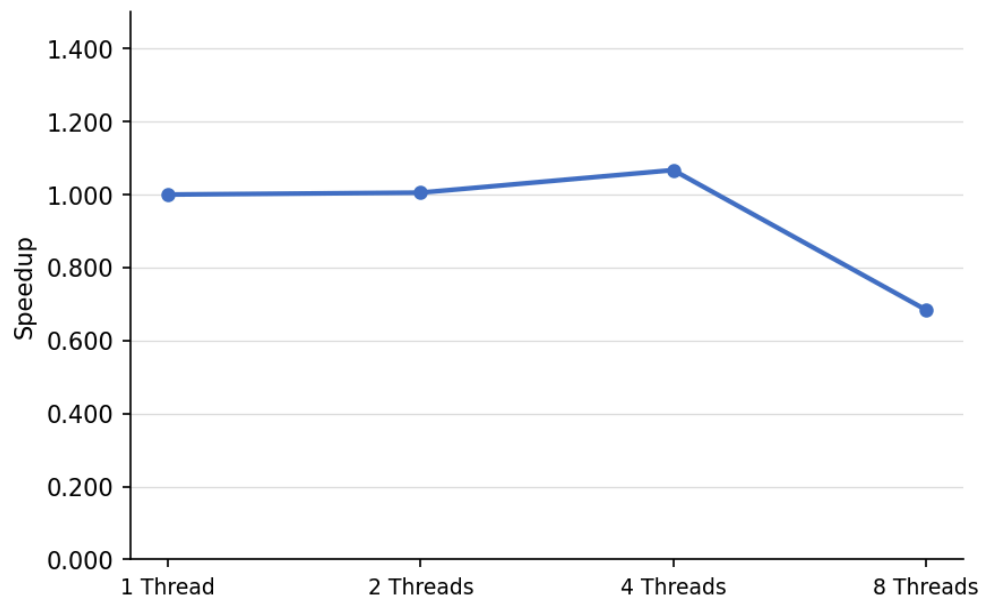
- Tabela Distribuído

Threads	Tempo* (s)	Speedup S = $T(1)/T(p)$	Eficiência E = S/p
1	3,0543	1	1
2	2,1251	1,4372	0,719
4	1,9962	1,5301	0,383
8	3,1656	0,9648	0,121

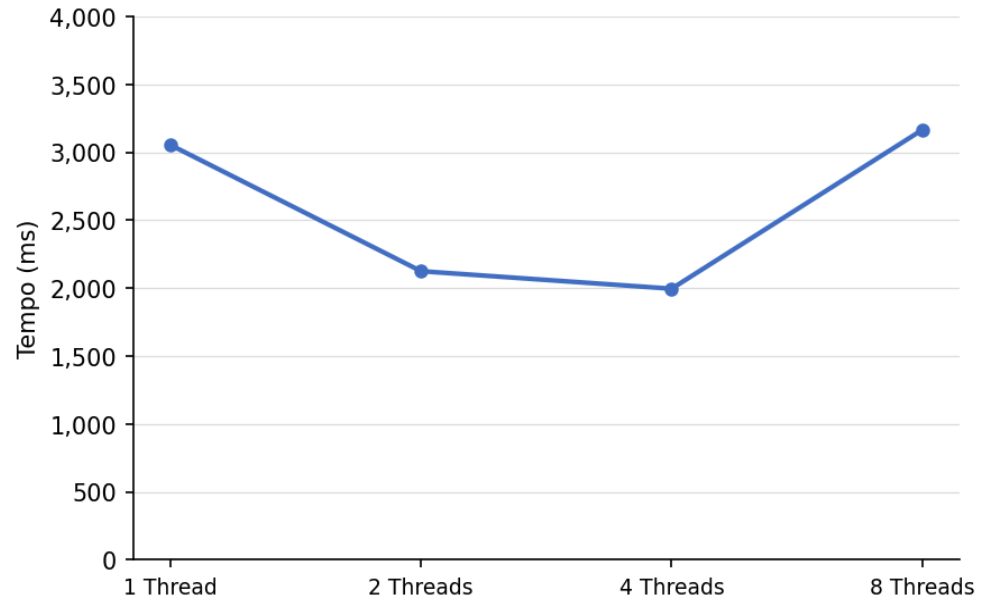
4.3.5. Gráficos

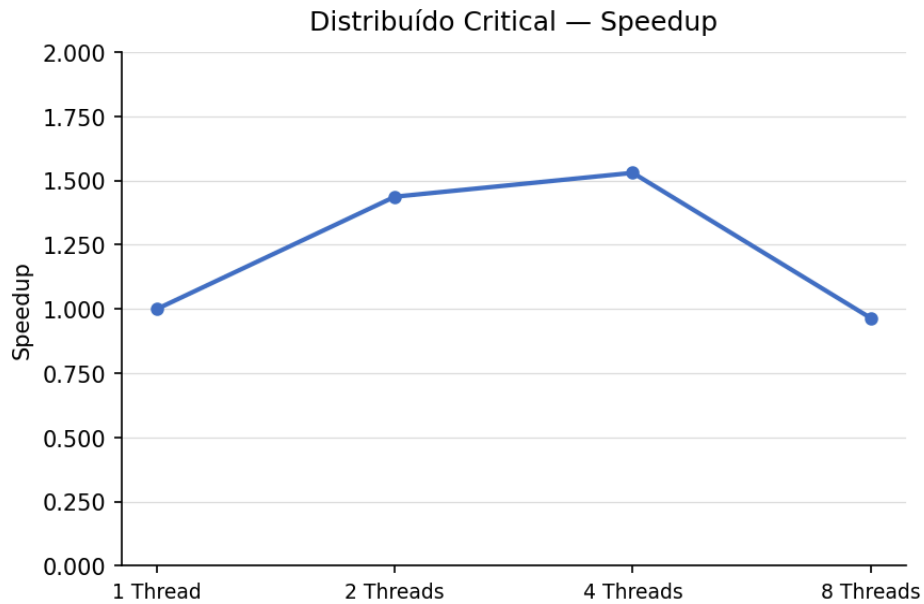


Concorrente Critical — Speedup



Distribuído Critical — Tempo





4.3.6. Análise Crítica e Conclusão Critical

A versão Critical apresentou a pior escalabilidade entre todas as abordagens analisadas no projeto. No distribuído, o speedup cresceu até 4 threads (1,53x), mas com 8 threads, ele acabou regredindo para 0,96x, ficando mais lento que até a execução com apenas 1 thread. No Concorrente, ele foi ainda mais limitado, ganhou no máximo apenas 1,07x em 4 threads, caindo para 0,68x quando usado 8 threads.

Esse padrão de degradação pode ser explicado pela natureza do portão global, com 8 threads, a maioria delas acaba passando mais tempo bloqueada aguardando a liberação do mutex do que processando as requisições em si. O user time de 17,291s com 8 threads no Concorrente confirma que as threads estavam ativas mas improdutivas, consumindo CPU sem fazer nada.

A análise de cache via Cachegrind revelou D1 miss rate de 0,6% no Concorrente e 1,0% no Distribuído, indicando que o gargalo do Critical não está no comportamento de cache, mas exclusivamente na serialização imposta pelo portão de sincronização.

Ou seja, se conclui que o `#pragma omp critical`, por sua granularidade grossa, é inadequado para aplicações com alta taxa de atualização de estruturas compartilhadas. Em ambos os modelos de acesso, a partir de 8 threads, o overhead de sincronização supera os benefícios do paralelismo.

5. Versão Paralela - Lock

5.1. Metodologia

A versão Lock utiliza locks individuais por bucket da tabela hash, por meio da API `omp_lock_t` do OpenMP. Nessa abordagem, em vez de uma única região crítica global como no **#pragma omp critical**, cada bucket possui seu próprio lock. Assim, threads que acessam buckets diferentes podem executar simultaneamente, sem se bloquear mutuamente. A contenção só ocorre quando duas ou mais threads tentam acessar o mesmo bucket ao mesmo tempo.

O fluxo de execução é: calcular o bucket da URL usando a mesma função hash (djb2) utilizada internamente pelo `hash_table.c`, adquirir o lock daquele bucket com `omp_set_lock`, incrementar o `hit_count`, e liberar o lock com `omp_unset_lock`.

Assim como nas demais versões, foram testados os dois modelos: Distribuído, com acessos espalhados entre URLs distintas, e Concorrente, com 90% do tráfego concentrado em poucas URLs.

5.2. Ambiente Utilizado

Máquina Usada: Máquina Linux Ubuntu 22.04.5 LTS, Core i7-1255U, 12 threads, 8GB RAM

Compilador: GCC e flag `-O2`

Ferramentas: `time`, `cachegrind`

5.3. Lock

5.3.1. Introdução ao Lock

Para a execução do Lock, utiliza-se a API `omp_lock_t` do OpenMP para proteger o acesso à tabela hash por meio de locks individuais por bucket. Ao contrário do **#pragma omp critical**, que serializa todas as threads em uma única região global, essa abordagem permite que threads acessando buckets diferentes executem simultaneamente, reduzindo a contenção e aumentando o paralelismo efetivo.

5.3.2. Execução e Testes → Concorrente

Threads: 1

```

aluno@206:~/Projeto2-Computacao-Paralela$ export OMP_NUM_THREADS=1
aluno@206:~/Projeto2-Computacao-Paralela$ gcc -O2 -fopenmp analyzer_par_lock.c hash_table.c -o analyzer_par_lock
aluno@206:~/Projeto2-Computacao-Paralela$ time ./analyzer_par_lock log_concorrente.txt
bash: time: ./analyzer_par_lock: Arquivo ou diretório inexistente
aluno@206:~/Projeto2-Computacao-Paralela$ time ./analyzer_par_lock log_concorrente.txt
Iniciando análise paralela (lock por bucket):
Tempo: 2.4572 segundos

real    0m2,505s
user    0m1,789s
sys     0m0,537s
aluno@206:~/Projeto2-Computacao-Paralela$ sort results.csv > results_concorrente_sorted.csv
aluno@206:~/Projeto2-Computacao-Paralela$ diff -s results_concorrente_sorted.csv gabarito_concorrente.csv
Os arquivos results_concorrente_sorted.csv e gabarito_concorrente.csv são idênticos
aluno@206:~/Projeto2-Computacao-Paralela$ date
ter 19 mai 2026 11:31:06 -03

```

Threads: 2

```

ter 19 mai 2026 11:31:06 -03
aluno@206:~/Projeto2-Computacao-Paralela$ export OMP_NUM_THREADS=2
aluno@206:~/Projeto2-Computacao-Paralela$ gcc -O2 -fopenmp analyzer_par_lock.c hash_table.c -o analyzer_par_lock
aluno@206:~/Projeto2-Computacao-Paralela$ time ./analyzer_par_lock log_concorrente.txt
Iniciando análise paralela (lock por bucket):
Tempo: 1.9691 segundos

real    0m2,018s
user    0m2,282s
sys     0m0,533s
aluno@206:~/Projeto2-Computacao-Paralela$ sort results.csv > results_concorrente_sorted.csv
aluno@206:~/Projeto2-Computacao-Paralela$ diff -s results_concorrente_sorted.csv gabarito_concorrente.csv
Os arquivos results_concorrente_sorted.csv e gabarito_concorrente.csv são idênticos
aluno@206:~/Projeto2-Computacao-Paralela$ date
ter 19 mai 2026 11:32:32 -03

```

Threads: 4

```

ter 19 mai 2026 11:32:32 -03
aluno@206:~/Projeto2-Computacao-Paralela$ export OMP_NUM_THREADS=4
aluno@206:~/Projeto2-Computacao-Paralela$ gcc -O2 -fopenmp analyzer_par_lock.c hash_table.c -o analyzer_par_lock
aluno@206:~/Projeto2-Computacao-Paralela$ time ./analyzer_par_lock log_concorrente.txt
Iniciando análise paralela (lock por bucket):
Tempo: 1.8351 segundos

real    0m1,882s
user    0m3,094s
sys     0m0,515s
aluno@206:~/Projeto2-Computacao-Paralela$ sort results.csv > results_concorrente_sorted.csv
aluno@206:~/Projeto2-Computacao-Paralela$ diff -s results_concorrente_sorted.csv gabarito_concorrente.csv
Os arquivos results_concorrente_sorted.csv e gabarito_concorrente.csv são idênticos
aluno@206:~/Projeto2-Computacao-Paralela$ date
ter 19 mai 2026 11:33:32 -03

```

Threads: 8

```

ter 19 mai 2026 11:33:32 -03
aluno@206:~/Projeto2-Computacao-Paralela$ export OMP_NUM_THREADS=8
aluno@206:~/Projeto2-Computacao-Paralela$ gcc -O2 -fopenmp analyzer_par_lock.c hash_table.c -o analyzer_par_lock
aluno@206:~/Projeto2-Computacao-Paralela$ time ./analyzer_par_lock log_concorrente.txt
Iniciando análise paralela (lock por bucket):
Tempo: 1.7712 segundos

real    0m1,819s
user    0m4,621s
sys     0m0,526s
aluno@206:~/Projeto2-Computacao-Paralela$ sort results.csv > results_concorrente_sorted.csv
aluno@206:~/Projeto2-Computacao-Paralela$ diff -s results_concorrente_sorted.csv gabarito_concorrente.csv
Os arquivos results_concorrente_sorted.csv e gabarito_concorrente.csv são idênticos
aluno@206:~/Projeto2-Computacao-Paralela$ date
ter 19 mai 2026 11:34:55 -03

```

Execução do Valgrind (4 Threads):

```

aluno@308:~/Projeto2-Computacao-Paralela$ gcc -O2 -fopenmp analyzer_par_lock.c hash_table.c -o analyzer_par_lock
aluno@308:~/Projeto2-Computacao-Paralela$ valgrind -s --tool=cachegrind ./analyzer_par_lock log_concorrente.txt
valgrind: failed to start tool 'cachegrind' for platform 'amd64-linux': No such file or directory
aluno@308:~/Projeto2-Computacao-Paralela$ valgrind -s --tool=cachegrind ./analyzer_par_lock log_concorrente.txt
==13070== Cachegrind, a cache and branch-prediction profiler
==13070== Copyright (C) 2002-2017, and GNU GPL'd, by Nicholas Nethercote et al.
==13070== Using Valgrind-3.18.1 and LibVEX; rerun with -h for copyright info
==13070== Command: ./analyzer_par_lock log_concorrente.txt
==13070==
--13070-- warning: L3 cache found, using its data for the LL simulation.
Iniciando análise paralela (lock por bucket):
==13070== brk segment overflow in thread #1: can't grow to 0x485b000
==13070== (see section Limitations in user manual)
==13070== NOTE: further instances of this message will not be shown
Tempo: 68.5073 segundos
==13070==
==13070== I refs:      27,420,312,298
==13070== I1 misses:      2,231
==13070== L1i misses:      2,230
==13070== I1 miss rate:      0.00%
==13070== L1i miss rate:      0.00%
==13070==
==13070== D refs:      9,167,945,765 (5,951,530,932 rd + 3,216,414,833 wr)
==13070== D1 misses:      58,122,113 ( 36,701,873 rd + 21,420,240 wr)
==13070== L1d misses:      55,495,753 ( 34,143,559 rd + 21,352,194 wr)
==13070== D1 miss rate:      0.6% ( 0.6% + 0.7% )
==13070== L1d miss rate:      0.6% ( 0.6% + 0.7% )
==13070==
==13070== LL refs:      58,124,344 ( 36,704,104 rd + 21,420,240 wr)
==13070== LL misses:      55,497,983 ( 34,145,789 rd + 21,352,194 wr)
==13070== LL miss rate:      0.2% ( 0.1% + 0.7% )
aluno@308:~/Projeto2-Computacao-Paralela$

```

5.3.3. Execução e Testes → Distribuído

Threads: 1

```

ter 19 mai 2020 11:34:33 -03
aluno@206:~/Projeto2-Computacao-Paralela$ export OMP_NUM_THREADS=1
aluno@206:~/Projeto2-Computacao-Paralela$ gcc -O2 -fopenmp analyzer_par_lock.c hash_table.c -o analyzer_par_lock
aluno@206:~/Projeto2-Computacao-Paralela$ time ./analyzer_par_lock log_distribuido.txt
Iniciando análise paralela (lock por bucket):
Tempo: 2.8236 segundos

real    0m2.869s
user    0m2.373s
sys     0m0.486s
aluno@206:~/Projeto2-Computacao-Paralela$ sort results.csv > results_distribuido_sorted.csv
aluno@206:~/Projeto2-Computacao-Paralela$ diff -s results_distribuido_sorted.csv gabarito_distribuido.csv
Os arquivos results_distribuido_sorted.csv e gabarito_distribuido.csv são idênticos
aluno@206:~/Projeto2-Computacao-Paralela$ date
ter 19 mai 2020 11:37:26 -03

```

Threads: 2

```

ter 19 mai 2020 11:37:20 -03
aluno@206:~/Projeto2-Computacao-Paralela$ export OMP_NUM_THREADS=2
aluno@206:~/Projeto2-Computacao-Paralela$ gcc -O2 -fopenmp analyzer_par_lock.c hash_table.c -o analyzer_par_lock
aluno@206:~/Projeto2-Computacao-Paralela$ time ./analyzer_par_lock log_distribuido.txt
Iniciando análise paralela (lock por bucket):
Tempo: 2.5380 segundos

real    0m2.586s
user    0m3.655s
sys     0m0.448s
aluno@206:~/Projeto2-Computacao-Paralela$ sort results.csv > results_distribuido_sorted.csv
aluno@206:~/Projeto2-Computacao-Paralela$ diff -s results_distribuido_sorted.csv gabarito_distribuido.csv
Os arquivos results_distribuido_sorted.csv e gabarito_distribuido.csv são idênticos
aluno@206:~/Projeto2-Computacao-Paralela$ date
ter 19 mai 2020 11:39:14 -03

```

Threads: 4

```

ter 19 mai 2026 11:39:14 -03
aluno@206:~/Projeto2-Computacao-Paralela$ export OMP_NUM_THREADS=4
aluno@206:~/Projeto2-Computacao-Paralela$ gcc -O2 -fopenmp analyzer_par_lock.c hash_table.c -o analyzer_par_lock
aluno@206:~/Projeto2-Computacao-Paralela$ time ./analyzer_par_lock log_distribuido.txt
Iniciando análise paralela (lock por bucket):
Tempo: 1.8591 segundos

real    0m1,907s
user    0m3,621s
sys     0m0,470s
aluno@206:~/Projeto2-Computacao-Paralela$ sort results.csv > results_distribuido_sorted.csv
aluno@206:~/Projeto2-Computacao-Paralela$ diff -s results_distribuido_sorted.csv gabarito_distribuido.csv
Os arquivos results_distribuido_sorted.csv e gabarito_distribuido.csv são idênticos
aluno@206:~/Projeto2-Computacao-Paralela$ date
ter 19 mai 2026 11:40:12 -03

```

Threads: 8

```

ter 19 mai 2026 11:40:12 -03
aluno@206:~/Projeto2-Computacao-Paralela$ export OMP_NUM_THREADS=8
aluno@206:~/Projeto2-Computacao-Paralela$ gcc -O2 -fopenmp analyzer_par_lock.c hash_table.c -o analyzer_par_lock
aluno@206:~/Projeto2-Computacao-Paralela$ time ./analyzer_par_lock log_distribuido.txt
Iniciando análise paralela (lock por bucket):
Tempo: 1.7627 segundos

real    0m1,831s
user    0m6,252s
sys     0m0,505s
aluno@206:~/Projeto2-Computacao-Paralela$ sort results.csv > results_distribuido_sorted.csv
aluno@206:~/Projeto2-Computacao-Paralela$ diff -s results_distribuido_sorted.csv gabarito_distribuido.csv
Os arquivos results_distribuido_sorted.csv e gabarito_distribuido.csv são idênticos
aluno@206:~/Projeto2-Computacao-Paralela$ date
ter 19 mai 2026 11:41:03 -03

```

Execução do Valgrind (4 Threads):

```

aluno@308:~/Projeto2-Computacao-Paralela$ gcc -O2 -fopenmp analyzer_par_lock.c hash_table.c -o analyzer_par_lock
aluno@308:~/Projeto2-Computacao-Paralela$ export OMP_NUM_THREADS=4
aluno@308:~/Projeto2-Computacao-Paralela$ valgrind -s --tool=cachgrind ./analyzer_par_lock log_distribuido.txt
valgrind: failed to start tool 'cachgrind' for platform 'amd64-linux': No such file or directory
aluno@308:~/Projeto2-Computacao-Paralela$ valgrind -s --tool=cachegrind ./analyzer_par_lock log_distribuido.txt
==12640== Cachegrind, a cache and branch-prediction profiler
==12640== Copyright (C) 2002-2017, and GNU GPL'd, by Nicholas Nethercote et al.
==12640== Using Valgrind-3.18.1 and LibVEX; rerun with -h for copyright info
==12640== Command: ./analyzer_par_lock log_distribuido.txt
==12640==
--12640-- warning: L3 cache found, using its data for the LL simulation.
Iniciando análise paralela (lock por bucket):
==12640== brk segment overflow in thread #1: can't grow to 0x485b000
==12640== (see section Limitations in user manual)
==12640== NOTE: further instances of this message will not be shown
Tempo: 67.7537 segundos
==12640==
==12640== I  refs:      27,182,906,480
==12640== I1 misses:      2,230
==12640== L1i misses:      2,229
==12640== I1 miss rate:      0.00%
==12640== L1i miss rate:      0.00%
==12640==
==12640== D  refs:      9,103,824,135 (5,900,207,141 rd + 3,203,616,994 wr)
==12640== D1 misses:      99,677,697 ( 78,254,090 rd + 21,423,607 wr)
==12640== L1d misses:      60,540,172 ( 39,179,369 rd + 21,360,803 wr)
==12640== D1 miss rate:      1.1% ( 1.3% + 0.7% )
==12640== L1d miss rate:      0.7% ( 0.7% + 0.7% )
==12640==
==12640== LL refs:      99,679,927 ( 78,256,320 rd + 21,423,607 wr)
==12640== LL misses:      60,542,401 ( 39,181,598 rd + 21,360,803 wr)
==12640== LL miss rate:      0.2% ( 0.1% + 0.7% )
aluno@308:~/Projeto2-Computacao-Paralela$

```

5.4. Tabelas

-
- Tabela Concorrente

Threads	Tempo* (s)	Speedup S = T(1)/T(p)	Eficiência E = S/p
1	2,4572	1	1
2	1,9691	1,2479	0,624

4	1,8351	1,3390	0,335
8	1,7712	1,3874	0,173

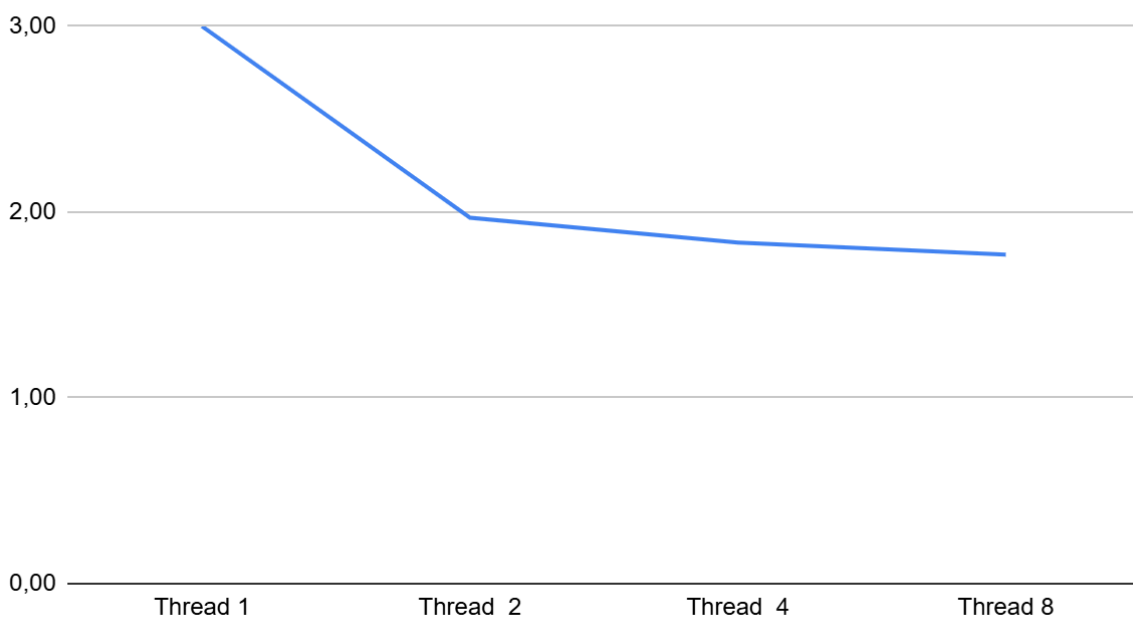
- Tabela Distribuído

Threads	Tempo* (s)	Speedup S = $T(1)/T(p)$	Eficiência E = S/p
1	2,8236	1	1
2	2,5380	1,1125	0,556
4	1,8591	1,5188	0,380
8	1,7627	1,6019	0,200

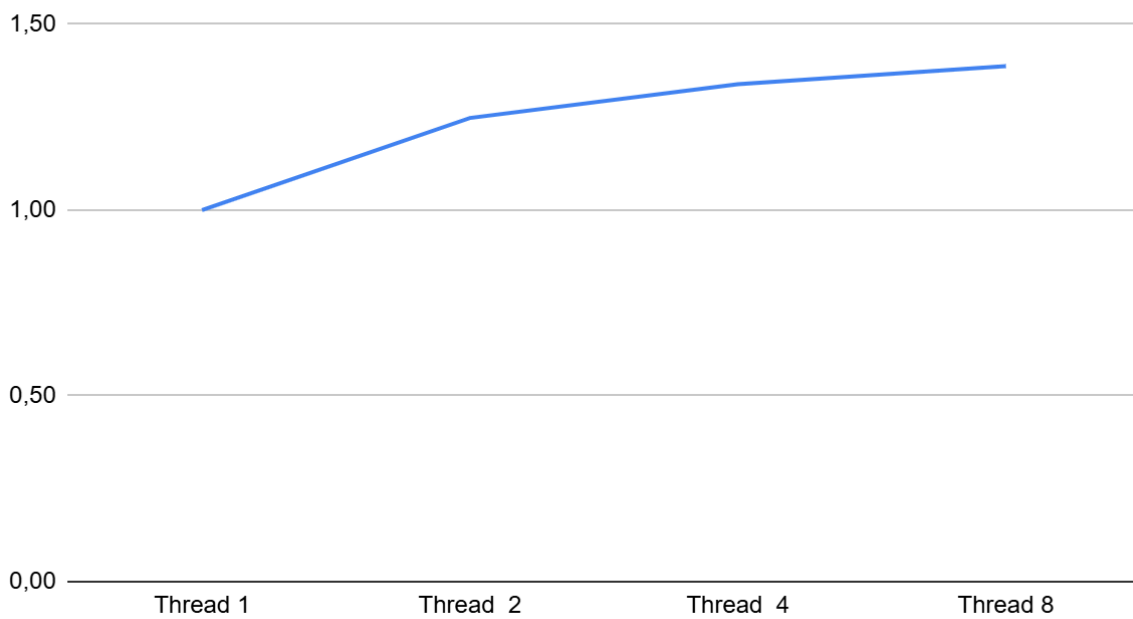
5.5. Gráficos

- Concorrente *Lock*

Tempo (ms)

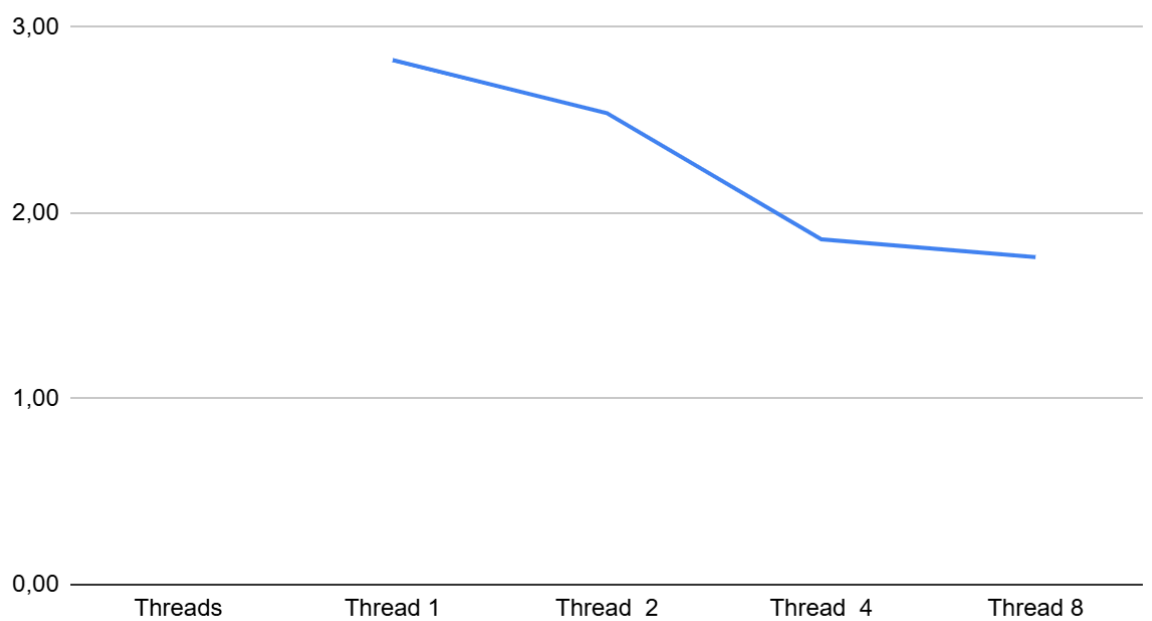


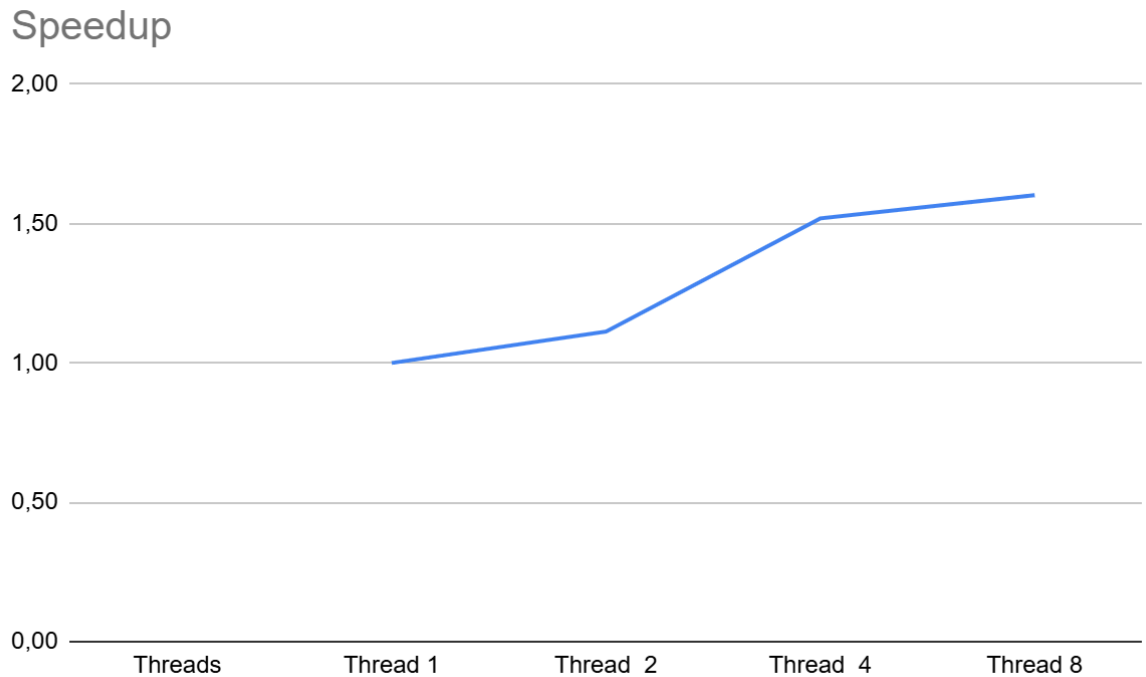
Speedup



- Distribuído Lock

Tempo(ms)





5.6.1. Análise Crítica Lock

A versão Lock apresentou ganhos de desempenho com o aumento de threads em ambos os datasets, porém com speedup modesto em relação ao esperado teoricamente. No modelo Concorrente, o speedup máximo com 8 threads foi de 1,3874, com eficiência de 0,173. Isso indica que, mesmo com locks individuais por bucket, a alta concentração de acessos em poucas URLs gera disputa frequente pelo mesmo lock, reduzindo o paralelismo efetivo.

5.6.2. Conclusão

Dessa forma, observa-se que, embora os locks por bucket reduzam a contenção global em relação ao `#pragma omp critical`, o overhead de aquisição e liberação dos locks continua limitando a escalabilidade da aplicação, especialmente em cenários de alta concentração de acessos.

5.6.3. Conclusão Geral

Comparando a versão Lock com as demais abordagens paralelas, observa-se que ela apresentou melhora no tempo de execução em relação à execução sequencial, demonstrando os benefícios do paralelismo com OpenMP. Entretanto, houve perda progressiva de eficiência com o aumento do número de threads, indicando limitações de escalabilidade associadas ao custo dos locks e à contenção em buckets muito acessados.

A versão Lock oferece granularidade intermediária de sincronização, sendo mais paralela que o `#pragma omp critical` e mais simples de implementar que abordagens lock-free. Assim, conclui-se que o uso de locks por bucket é eficiente para reduzir contenção global, mas ainda apresenta limitações em cenários de alta concorrência onde poucos buckets concentram a maior parte dos acessos.

6. Questões de Análise

6.1. Porque a versão critical pode degradar sob alta contenção?

Porque o `#pragma omp critical` usa um único lock global pro programa todo. Ou seja, quando ele tem 8 threads no log concorrente, todas elas acabam disputando do mesmo portão independente da URL que estão acessando. Como é possível ver nos resultados: Um speedup de 0.68x com 8 Threads, é mais lento do que rodar com apenas 1 thread, o user time de 17,291s comparado ao real de 3,060s mostra que as threads estavam acordadas mas paradas na fila, consumindo CPU sem fazer nada.

6.2. Como locks por buckets equilibram sincronização e paralelismo?

Em vez de um único lock global como no `#pragma omp critical`, a versão Lock cria um lock independente para cada bucket da tabela hash. Assim, threads que acessam URLs mapeadas para buckets diferentes podem executar ao mesmo tempo, sem se bloquear. A contenção só ocorre quando duas threads tentam atualizar URLs do mesmo bucket simultaneamente. Nos resultados, isso se reflete em um speedup de 1,6019 com 8 threads no Distribuído, superior ao 0,9648 do Critical no mesmo cenário, justamente porque no Distribuído as URLs se espalham por buckets diferentes, reduzindo a disputa.

6.3. Compare cache misses entre versões atomic e padded.

Comparando os valores de cache miss entre a versão atomic e padded, é notável que a versão padded tende a ter menos cache misses do que a atomic, principalmente em L1d e LLd, o que sugere menos false sharing e melhor isolamento entre threads. Portanto, padded sofre menos com contenção de cache que a versão atomic já que o padding separa os elementos na cache line e reduz a interferência entre threads, diminuindo misses.

6.4. Houve speedup superlinear?

Não houve speedup superlinear em nenhuma das versões analisadas. O speedup máximo observado foi de 2,0042 na versão Atomic Distribuído com 8 threads, valor próximo ao linear mas não superior. Para ocorrer speedup superlinear seria necessário que o programa paralelo fosse mais rápido do que o sequencial dividido pelo número de threads, o que normalmente exige efeitos de cache favoráveis, como dados que cabem na cache de múltiplos núcleos mas não na cache de um único núcleo. Neste projeto, o tamanho da tabela hash e o volume de dados não favoreceram esse comportamento.

6.5. Quando o aumento de memória causado pelo padding torna-se vantajoso?

Quando muitas threads acessam variáveis da mesma cache line, especialmente quando cada thread atualiza seu próprio contador. Nestes casos, sem o padding, uma thread invalida a linha de cache usada por outra, forçando recargas constantes e aumentando os misses, o que caracteriza *false sharing*. O padding, portanto, separa dados que são utilizados por threads diferentes, evitando que eles compartilhem a mesma cache line.

6.6. Qual solução seria utilizada em um CDN real?

Em um CDN real, a solução mais adequada seria uma variação da versão Lock, porém com otimizações adicionais. O `#pragma omp critical` seria descartado por sua serialização global. O Atomic é eficiente para operações simples, mas pode sofrer contenção em hotspots. O Lock por bucket oferece granularidade intermediária e boa escalabilidade. Na prática, sistemas reais como o Nginx e bancos de dados de alto desempenho utilizam estruturas semelhantes, combinando locks por partição com técnicas lock-free para os caminhos mais críticos. Para o cenário do `log_concorrente`, onde poucos conteúdos concentram a maioria dos acessos, seria necessário também um balanceamento melhor da função hash para distribuir esses hotspots entre mais buckets.

6.7. Como listas encadeadas impactam localidade de cache?

Listas encadeadas prejudicam a localidade de cache porque seus nós são alocados dinamicamente em posições não contíguas da memória. Quando uma thread percorre a lista de um bucket para encontrar uma URL, cada acesso ao próximo nó pode resultar em um cache miss, pois o nó seguinte provavelmente não está na cache. Isso é confirmado pelos valores de D1 miss rate observados no Cachegrind, em torno de 0,6% a 1,2% dependendo da versão. Em comparação, estruturas contíguas como arrays teriam muito melhor localidade. No contexto deste projeto, o impacto é mais severo no `log_distribuido`, onde acessos a URLs variadas forçam a percorrerência de listas em buckets diferentes com frequência.

7. Conclusão

Este projeto permitiu comparar na prática quatro estratégias de sincronização em OpenMP para o processamento paralelo de logs de acesso de uma CDN. A versão sequencial serviu como baseline, apresentando execução estável e previsível. Entre as versões paralelas, o `#pragma omp critical` demonstrou a pior escalabilidade, chegando a ser mais lento que a execução com 1 thread em cenários de alta contenção com 8 threads. A versão Atomic apresentou bom desempenho para operações simples de incremento, com ganhos consistentes de speedup, porém com queda progressiva de eficiência. O Atomic Padded reduziu parcialmente os efeitos de false sharing, sem eliminar completamente o overhead atômico. A versão Lock por bucket ofereceu o melhor equilíbrio entre segurança de concorrência e paralelismo efetivo, mantendo speedup crescente sem a degradação observada no Critical. Conclui-se que a escolha da estratégia de sincronização impacta diretamente a escalabilidade do sistema, e que granularidade fina de sincronização, como locks por bucket, é mais adequada para aplicações com alta taxa de atualização de estruturas compartilhadas.